VIT®
Vellore Institute of Technology
(Deemed to be University under section 3 of UGC Act, 1956)

M.Tech – Computer Science and Engineering
MACSE698 – Internship I / Dissertation I

Quantum-Enhanced Vision-Based Pollen Deficiency Estimation for Precision Sunflower Pollination

Team members:
Ramit Bose 25MCS0029

Faculty guide :
Dr. Samriddhi Sarkar

Approval Mail From Guide

- *Get the approval for the following points from your guide through mail before 01-09-2026.(sample attached below)*

The **Fall Semester Project Review 2 for Project ID <<25000UG01>> has been approved.**

- **Meeting with Guide Frequently:** Satisfied / ~~Not Satisfied~~
- **PowerPoint Presentation:** Approved / ~~Not Approved~~
- **Project Demonstration:** Approved /~~ Not Approved~~
- **Project Report Draft:** Satisfied / ~~Not Satisfied~~
- **Project Outcome:** ~~Ongoing~~ / Completed

All the above aspects are addressed and updated by the project team.

Gmail

samriddhi

Active

Compose

Inbox 2,748

Starred

Snoozed

Sent

Drafts 3

Categories

Purchases

More

Labels

Request for Approval - Review 2

Inbox ×

Ramit Bose 25MCS0029 Tue, Sep 29, 11:29 AM (23 hours ago)

I have attached the PPT and Report for the upcoming review. Review 2 QIP Report 25MCS0025.docxName: Ramit Bose Reg No: 25MCS0029 ramit.bose2025@vitstude…

Samriddhi sarkar Tue, Sep 29, 12:29 PM (22 hours ago)

to me

Title finalized and updated on Vtop:Yes
PPT approved: Yes
Guide Reviewed date and time:29/09/2026 at 3pm
Meeting with Guide Frequently: Satisfactory
Remarks: Approved

On Tue, Sep 29, 2026 at 11:59 AM Samriddhi sarkar <samriddhi.sarkar@vit.ac.in> wrote:

> ...
>
> ***Disclaimer:***
> *This message was sent from Vellore Institute of Technology. The contents of this email may contain legally protected confidential or privileged information of "Vellore Institute of Technology". If you are not the intended recipient, you should not disseminate, distribute or copy this e-mail. Please notify the sender immediately and destroy all copies of this message and any attachments. If you have received this email in error, please promptly notify the sender by reply email and delete the original email and any backup copies without reading them.*

Disclaimer:
This message was sent from Vellore Institute of Technology. The contents of this email may contain legally protected confidential or privileged information of "Vellore Institute of Technology". If you are not the intended recipient, you should not disseminate, distribute or copy this e-mail. Please notify the sender immediately and destroy

Reply Forward Share in chat New

Aim

- To build a smart, low-cost system that looks at photos of sunflower fields and figures out which areas are not being pollinated well.
- **Step 1 – Look at the flowers:** use sunflower photos to tell which flowers are healthy and blooming well, and which seem to lack pollination.
- **Step 2 – Double-check with a newer technique:** use an experimental quantum-computing method to see whether it can make the assessment more accurate.
- **Overall goal:** show that this kind of targeted farming support is possible on a normal laptop – no supercomputers, real drones or expensive lab equipment; everything is built and tested through simulation first.

Abstract

- **Problem:** sunflower yield depends on effective pollination during a short flowering period, but large-scale monitoring of pollination adequacy by manual inspection is difficult.
- **Phase 1 – Classical Vision:** a Pollen-Deficiency Index (PDI) is estimated from sunflower images using classical Digital Image Processing, segmentation and machine-learning classification.
- **Phase 2 – Quantum Enhancement:** the estimate is refined with a hybrid quantum–classical model (dimensionality reduction + Variational Quantum Circuit) under NISQ constraints; quanvolutional feature maps are planned next.
- **Measured on 5,350 real images:** RF/SVM baselines reach 78–91% test accuracy; a 6-qubit VQC reaches 90.9% vs 97.0% for an SVM on the same binary task.
- **Contribution:** a resource-efficient, reproducible pipeline linking quantum image processing and precision agriculture, run locally on a MacBook Air M2 (16 GB).

Literature Review (1/2)

#	Paper Name	Objective	Methodology	Pros/Cons	Findings	Reference
1	**Sunflower-YOLO: Detection of sunflower capitula in UAV remote sensing images**	Detect sunflower capitula across multiple phenological states in UAV RGB imagery.	Constructed a multi-phenological dataset of 1,539 UAV images and 75,846 instances; applied augmentation; introduced a lightweight YOLOv7-tiny-derived Sunflower-YOLO detector.	Pros: very large number of annotated instances; multi-phenological design; lightweight architecture suitable for edge deployment. Cons: object detection does not provide pixel-level masks; performance may decline with severe overlap, occlusion, or different cultivars and altitudes.	Achieved mAP@0.5 of approximately 93%. Provides one of the strongest direct baselines for sunflower-head localization from UAV images.	Jing, R., Niu, Q., Tian, Y., Zhang, H., Zhao, Q., Li, Z., Zhou, X., & Li, D. (2024). European Journal of Agronomy, 160, 127332. DOI: 10.1016/j.eja.2024.127332 [1]
2	**Remote Phenotyping Strategies for Sunflower Flowering Assessments Using Deep Learning Approaches**	Count sunflower heads from UAV RGB imagery and estimate the 50% flowering date.	Deep-learning flower-head detection on repeated UAV flights; flower counts modelled over time using a logistic function; results compared with ground observations.	Pros: directly links UAV detection with a biologically meaningful flowering metric; temporal modelling uses only a small number of flights; non-destructive and scalable. Cons: preprint status; performance depends on flight timing, head visibility, and detection quality; flowering date is not equivalent to successful pollination.	Estimated flowering date correlated strongly with ground measurements (r>0.91). Five flights/data points were sufficient to estimate 50% flowering.	Authors not fully recoverable from the indexed record (2025). bioRxiv preprint. DOI: 10.1101/2025.04.03.646664 [8]
3	**Evaluating Transformer- and CNN-Based Semantic Segmentation Models for Sunflower Inflorescence Identification Using a UAV RGB Orthomosaic**	Compare transformer and CNN semantic-segmentation models for sunflower-inflorescence identification.	UAV RGB orthomosaic acquired during the seed-filling/near-maturity period; 492 image patches with binary masks and 24,912 annotated inflorescence instances; evaluated transformer and CNN segmentation architectures.	Pros: pixel-level masks; explicit architecture comparison; large number of inflorescence instances; directly relevant to flower-head area estimation. Cons: imagery collected after flowering, limiting active-pollination assessment; one acquisition date and site restrict generalization.	Shows feasibility of mapping sunflower inflorescences from UAV orthomosaics and provides a recent benchmark for segmentation-model selection.	Yildirim, E., Colkesen, I., & Sefercik, U.G. (2026). Precision Agriculture, 27, 56. DOI: 10.1007/s11119-026-10362-5 [2]
4	**Sunpheno: A Deep Neural Network for Phenological Classification of Sunflower Images**	Automate sunflower phenological-stage classification and reduce subjectivity in visual scoring.	Built an expert-labelled database of 5,000 field images; evaluated five deep-learning methods using transfer learning; ResNet50-based model selected for accuracy and speed; applied model to contrasting sunflower lines during senescence.	Pros: sizeable labelled dataset; transfer learning; connects phenology with senescence and yield-related interpretation. Cons: images captured with cell phones rather than UAVs; high-confidence misclassifications occurred; genotype and season transferability require further testing.	ResNet50 outperformed the alternative methods in accuracy and speed. Sunpheno reproduced developmental progression across seasons and genotypes, but confidence calibration remains important.	Luoni, S.B. et al. (2024). Plants, 13(14), 1998. DOI: 10.3390/plants13141998 [3]
5	**Automated Sunflower Head Detection and Yield Estimation from High-Resolution UAV Imagery Using YOLOv11**	Detect sunflower heads and use detections for scalable yield estimation.	Used 215 UAV images divided into 1,290 tiles; 80:20 train/test split; YOLOv11 trained and evaluated while tuning confidence and IoU thresholds.	Pros: recent detector; connects perception to yield; reports precision, recall, mAP, and F1; scalable workflow. Cons: yield estimation is indirect; no pollination-status label; results depend on the reported field and cultivar.	At confidence threshold 0.50 and IoU threshold 0.40: precision 0.84, recall 0.95, mAP@0.5 0.95, F1 0.90.	Iamchuen, N., Hongpradit, P., Puttinaovarat, S., & Anucharn, T. (2026). Sustainability, 18(2), 1026. DOI: 10.3390/su18021026 [9]

Literature Review (2/2)

#	Paper Name	Objective	Methodology	Pros/Cons	Findings	Reference
6	**Hybrid Quantum-Classical-Quantum Convolutional Neural Networks**	Improve image classification expressivity by combining quantum convolution, classical CNN layers, and a trainable VQC.	Three stages: fixed quantum convolution filter, shallow classical CNN, and trainable QNN classifier using ZZFeatureMap and RealAmplitudes; evaluated on MNIST, Fashion-MNIST, and MRI images with noise and finite shots.	Pros: trainable quantum decision boundary; classical CNN reduces dimensionality and qubit requirements; includes depolarizing-noise and finite-shot tests. Cons: small benchmark datasets; fixed quantum filter may limit adaptation; accuracy gains may not transfer to complex UAV scenes.	Reported competitive accuracy and convergence against classical and hybrid baselines. Especially relevant to NISQ sunflower work because classical feature extraction reduces the quantum input dimension.	Long, C., et al. (2025). Scientific Reports, 15. DOI: 10.1038/s41598-025-13417-1 [4]
7	**A Novel Quantum Convolutional Neural Network Framework for Quantum-Enhanced Classification of Pixelated Colour Images**	Design a low-resolution colour-image QCNN for binary and multiclass NISQ classification.	Used a problem-specific RGB/spatial block-matrix encoding, hierarchical ZZFeatureMap, quantum convolution-pooling pipeline, and COBYLA-optimized VQC; evaluated on a small six-class colour-image dataset.	Pros: colour-aware encoding; end-to-end quantum convolution and pooling; designed for low qubit/depth conditions. Cons: only 50 images in the reported six-class experiment; small-data results may be unstable; requires independent reproduction.	Reported 82.05% validation accuracy versus 40.00% for a classical CNN on the six-class task, suggesting potential generalization benefits in very small datasets.	Daka, C., et al. (2026). Scientific Reports. DOI: 10.1038/s41598-026-45140-w [5]
8	**Efficient Quantum Convolutional Neural Networks for Image Classification**	Reduce input dimensionality and design efficient QCNN building blocks for NISQ image classification.	Proposed an encoding that enables a primitive QCNN to process 28x28 MNIST using 49 qubits; selected blocks using expressibility, entanglement, and complexity; tested on IBM Heron r2 hardware.	Pros: addresses dimensionality and real-hardware execution; introduces circuit-selection criteria; directly relevant to few-qubit design. Cons: MNIST is simple compared with UAV images; 49 qubits is still substantial; reported advantage needs independent replication.	Reported 96.08% accuracy on IBM Heron r2 versus 71.74% for a classical benchmark under identical conditions.	Authors indexed under arXiv:2505.05957 (2025). DOI: 10.48550/arXiv.2505.05957 [10]
9	**Hybrid Quantum Neural Network for Rice Leaf Disease Classification**	Assess whether a hybrid CNN-quantum circuit improves agricultural disease-image classification.	Used 5,932 rice-leaf images covering four diseases; images standardized to 300x300; combined classical CNN representation with quantum circuits and benchmarked against CNN.	Pros: real agricultural image dataset; direct classical-versus-hybrid comparison; relevant workflow for sunflower health/flowering images. Cons: rice disease domain differs from sunflower pollination; reported metrics require peer-reviewed full-text verification; possible overfitting must be examined.	Reported best training accuracy of 99.93% and validation accuracy of 99.83% for the HQNN model.	Authors indexed as "Hybrid Quantum Neural Network for Rice Leaf Disease Classification" (2024). Verify venue, authors, and DOI from the publisher record. [11]
10	**Quanvolutional Neural Networks: Powering Image Recognition with Quantum Circuits**	Introduce quantum circuits as local convolution-like feature extractors for image recognition.	Split images into local patches, encode patches into small quantum circuits, measure quantum-generated features, and pass feature maps to classical layers.	Pros: uses few qubits per local patch; compatible with hybrid CNNs and NISQ devices; preserves spatial locality. Cons: fixed/random filters may not be trainable; repeated circuit execution over patches is costly; quantum advantage is not guaranteed.	Demonstrates that quanvolution can be integrated into classical image-recognition pipelines and provides a practical template for quantum-first image features.	Henderson, M., Shakya, S., Pradhan, S., & Cook, T. (2020). Quantum Machine Intelligence, 2, 2. DOI: 10.1007/s42484-020-00012-y [12]

Research Gap

1. No specific PDI for sunflower pollen deficiency.
2. No clear link between sunflower appearance and pollen deficiency.
3. No single system combining image segmentation, visual features, and deep features for PDI.
4. Limited study of individual sunflower parts at the pixel level during flowering.
5. Quantum methods have not been used to estimate sunflower pollen deficiency.
6. Limited fair comparison between quantum and classical models using the same features.
7. Few studies examine how quantum models handle noise in real agricultural images.
8. No complete classical–quantum system that predicts Low, Medium, or High pollen deficiency from sunflower images.

Objectives

- **O1:** Curate and verify a 5-class real sunflower dataset (5,350 images) – done
- **O2:** Classical DIP segmentation of the capitulum: CLAHE, denoising, GrabCut + K-means – done
- **O3:** Handcrafted features + rule-based PDI proxy labels (Low / Medium / High) – done
- **O4:** Classical baselines (Random Forest, SVM), 70/15/15 split – done
- **O5:** Hybrid 6-qubit VQC refinement compared with the classical baseline – done (proof of concept)
- **O6:** Noise-robustness and ablation studies – done (full per-qubit VQC ablation pending)

Why it is a Novelty?

1. **First-of-its-Kind Application:** It pioneers the use of **Quantum Machine Learning (QML)** for precision agriculture, moving crop monitoring beyond standard classical Deep Learning.

2. **Explainable Computer Vision:** Instead of a "black-box" AI, it uses a custom, transparent pipeline to extract 35 biologically meaningful features (like disc size and color) to calculate Pollen Deficiency.

3. **Hardware-Ready (Noise Tested):** It proves the quantum algorithm isn't just theoretical by simulating real-world hardware noise, showing the model is highly robust for today's early-stage quantum computers (NISQ devices).

Phase 1A: Color Space Conversion (RGB to Lab)

Input RGB
(200 × 200 pixels)

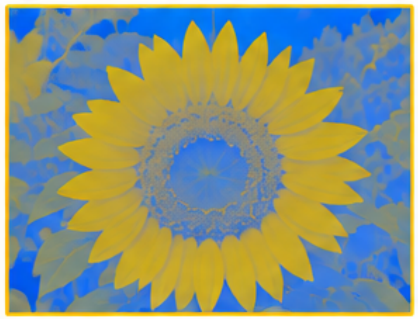
Convert RGB to Lab

L: Lightness
(Grayscale Detail)

- Represents brightness
- Contains structural details (edges, textures, shapes)
- Used for contrast enhancement, noise reduction and segmentation

a: Green-Red
color component

- Encodes green (negative values) to red (positive values)
- Captures chrominance information
- Helps distinguish flower from foliage



b: Blue-Yellow
color component

- Encodes blue (negative values) to yellow (positive values)
- Captures chrominance information
- Useful for separating petals (yellow) from background (blue/green)

How it works:

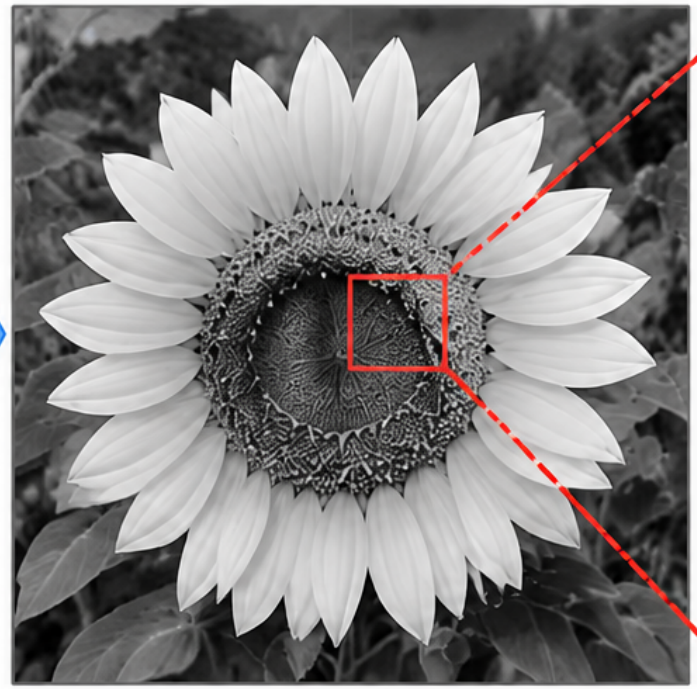
1. Start with the standardized RGB image (200 × 200 pixels).
2. Convert the image from the RGB color space to the **Lab** color space using cv2.cvtColor(img, cv2.COLOR_BGR2LAB).
3. This separates the **Lightness channel (L), which contains** structural details (brightness, textures, edges), from the chrominance channels (a and b), which contain pure color information (**green-red and blue-yellow**).
4. Isolating the **L** channel is crucial because it allows us to enhance contrast and perform segmentation based on structural information, independent of color variations due to lighting conditions.

Phase 1B: Input for Structural Analysis (Isolated L-Channel)

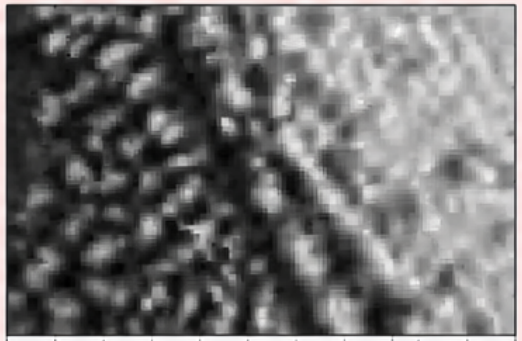
Input RGB Image
(200 × 200 pixels)

L-Channel Conversion

Isolated L-Channel Image (Grayscale)
(200 × 200 pixels, 8-bit, values 0–255)



Zoomed 10 × 10 Patch (Pixel Intensity Values)



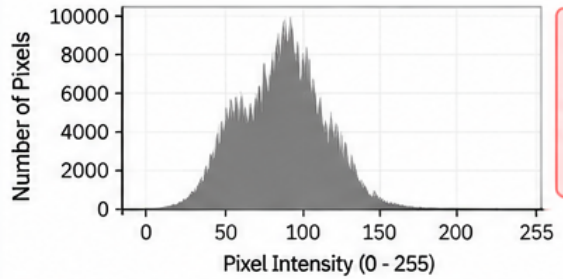
87	92	96	100	103	102	98	94	9	90	88
85	89	94	98	101	104	100	96	9	93	89
82	87	92	97	102	105	103	99	9	95	90
79	84	90	95	100	104	106	102	2	97	92
76	81	87	92	98	102	103	101	3	96	91
73	78	84	89	94	98	100	99	9	94	88
70	75	80	85	90	94	97	95	5	90	85
67	72	77	82	86	91	93	91	1	86	80
64	69	73	78	82	86	88	85	5	81	76
61	66	70	74	78	81	84	80	9	76	72

How it works:

1. Start with the standardized RGB image (200 × 200 pixels) from the dataset.
2. Convert the image from RGB to Lab color space and **extract** the **L-channel (Lightness)**.
3. The L-channel is a **single-channel grayscale image** with pixel **intensity** values ranging from 0 to 255.
4. This grayscale matrix strictly contains **structural information** such as **brightness, edges, textures and shapes,** independent of color information (a and b channels).
5. The L-channel matrix serves as the foundational input for **contrast enhancement (CLAHE)** and subsequent **segmentation.**

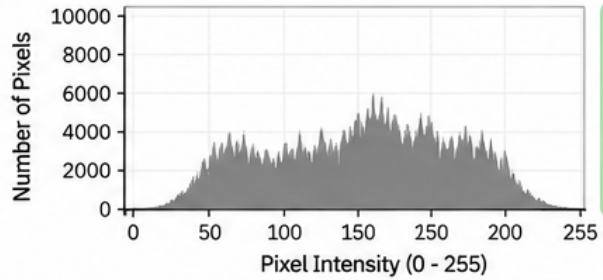
Phase 1C: Contrast Enhancement (CLAHE on L-Channel)

Histogram Before CLAHE (Uneven Contrast)



- Intensities concentrated in a narrow range
- Low contrast
- Shadows and textures are not clearly visible

Histogram After CLAHE (Enhanced Contrast)

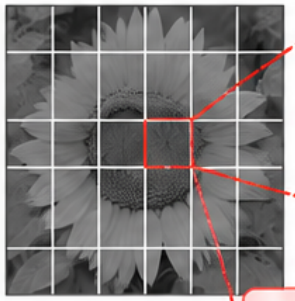


- Intensities widely distributed
- Higher contrast
- Structural details (textures, edges) are enhanced

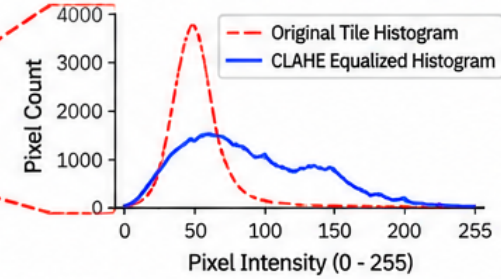
Input L-Channel (Before CLAHE)

CLAHE: Local Histogram Equalization

Divide into small tiles
(e.g., 8 × 8)



Histogram Equalization
on each tile (with contrast limiting)



Key Idea:
Enhance contrast locally in each tile while limiting noise amplification (contrast clipping).

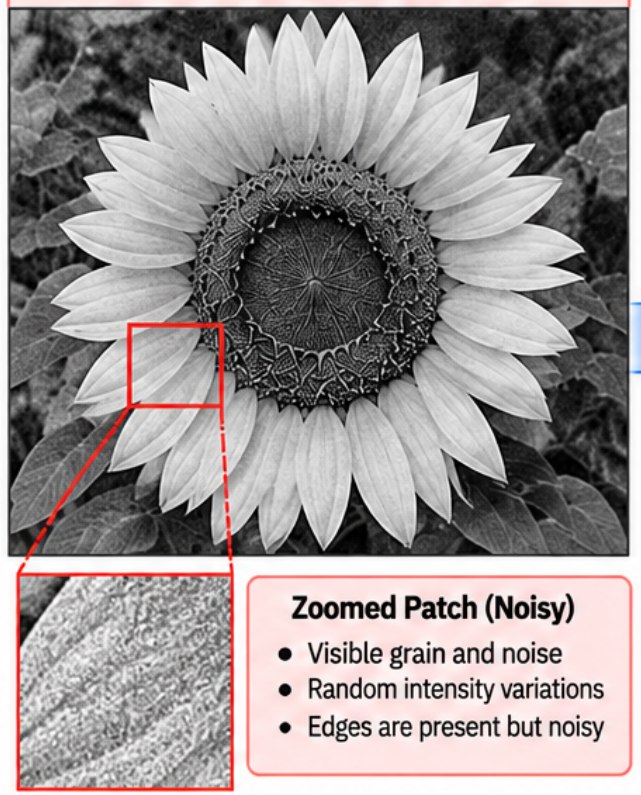
Enhanced L-Channel Output (After CLAHE)

How it works:

1. Start with the unenhanced L-channel grayscale image.
2. Apply **CLAHE** (Contrast Limited Adaptive Histogram Equalization) by dividing the image into small tiles (e.g., 8 × 8).
3. Perform histogram equalization on each tile to locally stretch the intensity values while limiting contrast to avoid noise amplification.
4. This locally increases contrast, making shadows on the sunflower disc and petals more consistent, regardless of lighting conditions.
5. Result: a high-contrast L-channel image with normalized illumination and clearly visible textures, ideal for the next segmentation step.

Phase 1D: Edge-Preserved Noise Reduction (Bilateral Filtering)

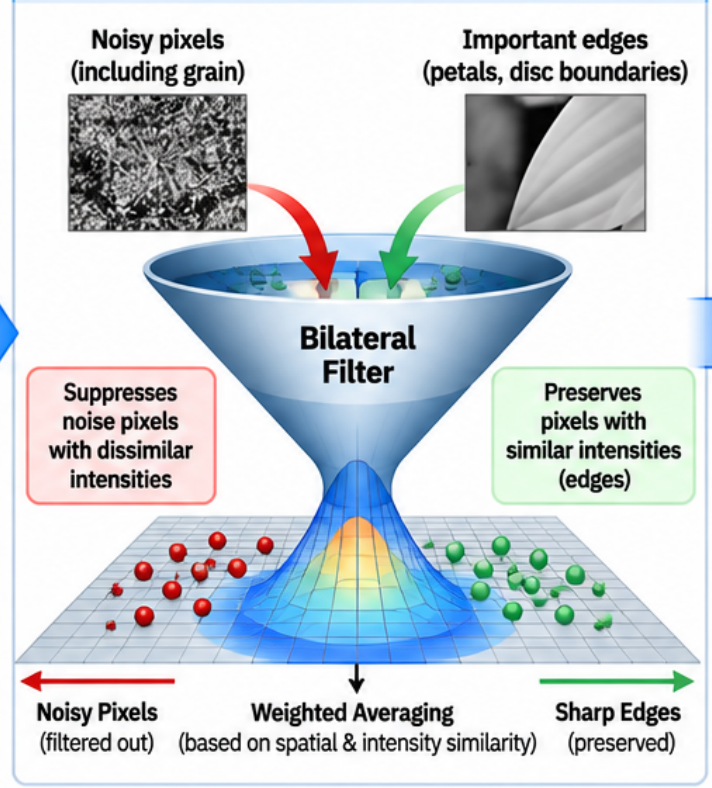
Input L-Channel (Noisy Image)
(After CLAHE, but with grain/noise)



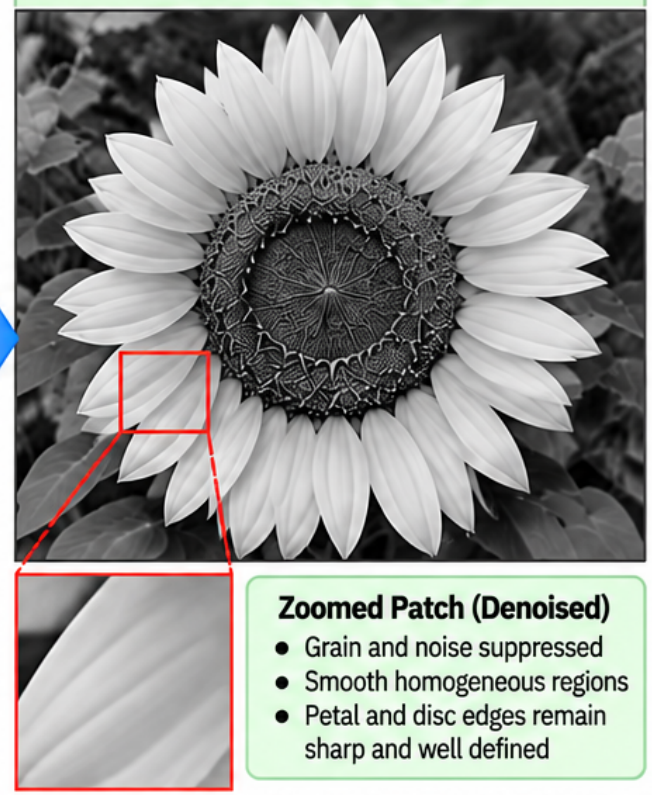
Zoomed Patch (Noisy)

- Visible grain and noise
- Random intensity variations
- Edges are present but noisy

Bilateral Filter
(Edge-Preserving Smoothing)



Output L-Channel (After Bilateral Filtering)
(Smooth but Sharp Edges)



Zoomed Patch (Denoised)

- Grain and noise suppressed
- Smooth homogeneous regions
- Petal and disc edges remain sharp and well defined

How it works:

1. Input the CLAHE-enhanced L-channel image, which may still contain grain and noise.
2. Apply a **Bilateral Filter**, which smooths the image by considering both spatial closeness and intensity similarity.
3. Pixels with similar intensities are averaged (noise is reduced), while pixels with large intensity differences (**edges**) are **preserved.**
4. This removes grain and unwanted noise while keeping critical structural edges (petals, disc boundaries) exceptionally sharp.
5. The resulting clean L-channel image is optimized for **robust segmentation** in the next phase.

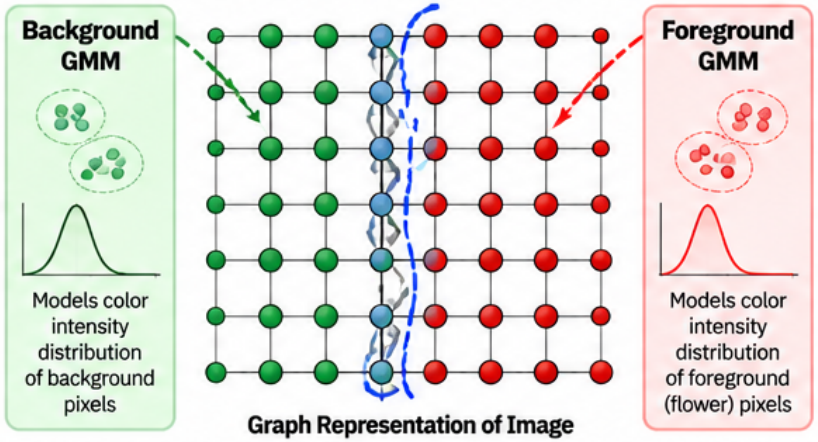
Phase 2E: Whole-Flower Segmentation (GrabCut)

1. Input Image with User-Defined Bounding Box
(Grayscale Image)

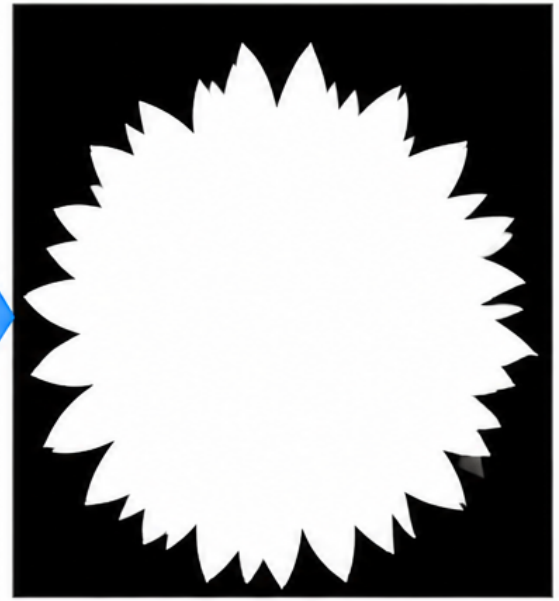
Input Grayscale Image
(with initial bounding box)

2. GrabCut: Unsupervised Graph-Based Optimization

Model each pixel using Gaussian Mixture Models (GMM) for foreground and background, and iteratively optimize a graph cut to separate them based on color/intensity similarity and spatial smoothness.



3. Output: Whole-Flower Mask (Binary Mask)



Result: Precise binary mask of the entire sunflower (petals + disc)

How it works:

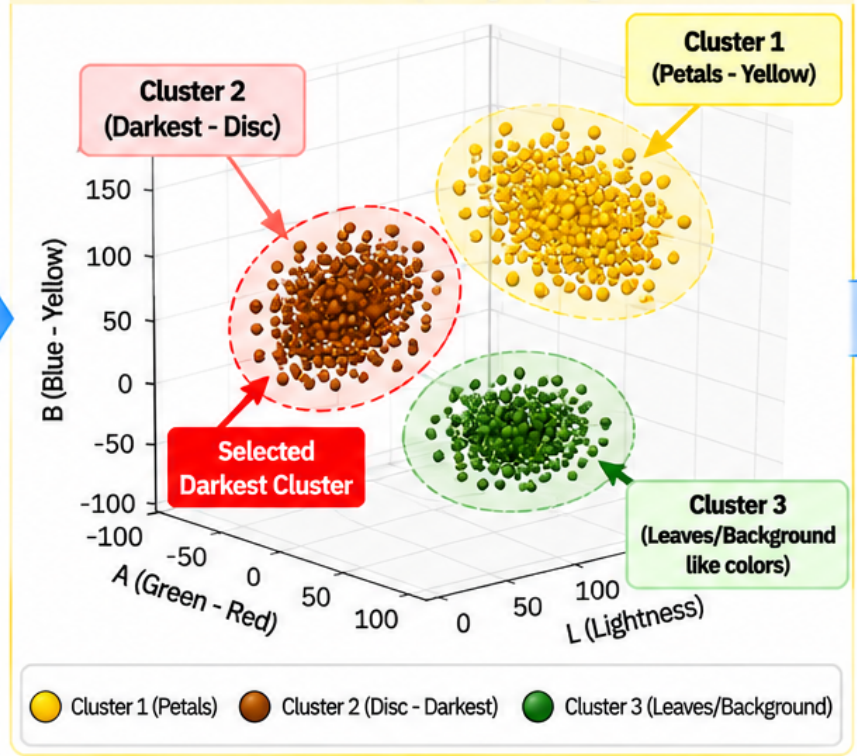
1. Start with the grayscale image and a user-defined bounding box around the flower.
2. GrabCut models the foreground (flower) and background using **Gaussian Mixture Models (GMM)** based on pixel color/intensity.
3. A graph is constructed where each pixel is a node, connected to its neighbors with edges that encode color similarity and spatial smoothness.
4. Using an iterative graph-cut optimization, the algorithm minimizes the energy function to separate foreground and background pixels.
5. Result: a precise binary mask of the entire sunflower (petals + disc), effectively cutting away the background and leaves.

Phase 2F: Unsupervised Disc Identification (K-Means Clustering)

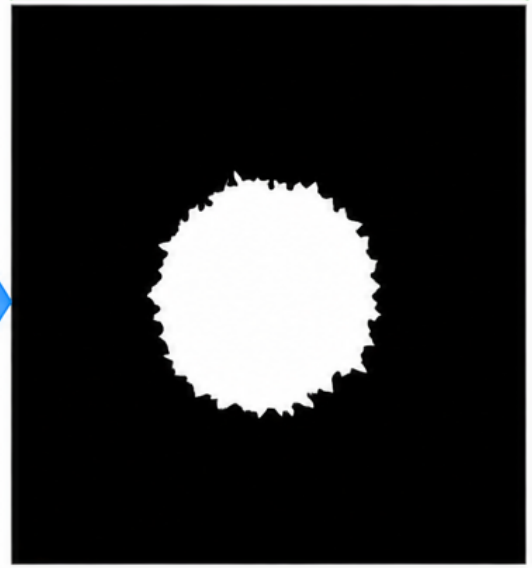
1. Input: Masked Sunflower (Fereground Only)
(Pixels inside whole-flower mask)

Input image (masked) containing only sunflower pixels (petals + disc)

2. K-Means Clustering (k = 3)
(Color-based pixel grouping)



3. Output: Disc Mask
(Binary Mask of Central Disc)



Isolated binary mask of the central pollen disc (white = disc, black = background)

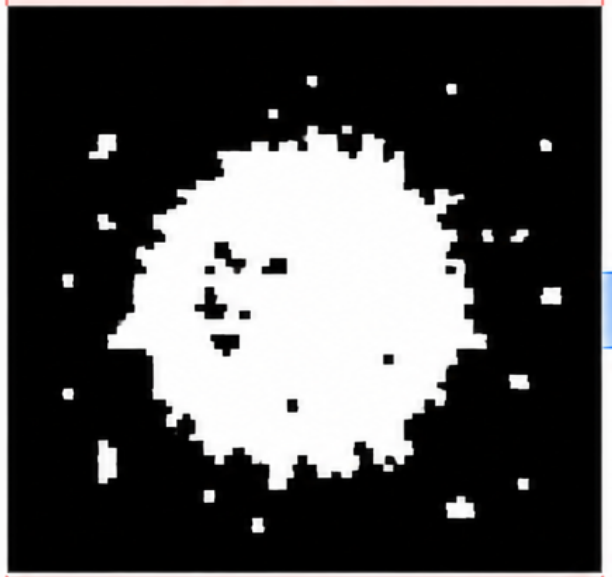
How it works:

1. Take only the foreground sunflower pixels from the whole-flower mask (petals + disc).
2. Convert the pixel colors to a suitable color space (e.g., Lab or RGB).
3. Apply K-Means clustering with k = 3 to group the pixels into 3 distinct color clusters.
4. The central pollen disc is consistently the darkest region, so the cluster with the lowest average lightness (darkest cluster) is automatically selected.
5. Generate a binary mask from this cluster, resulting in a precise isolation of the central disc area.
6. This unsupervised approach effectively separates the disc from the petals without using any pre-trained models.

Phase 2G: Mask Refinement (Mathematical Morphology)

1. Raw Disc Mask (From K-Means)

(Noisy binary mask)



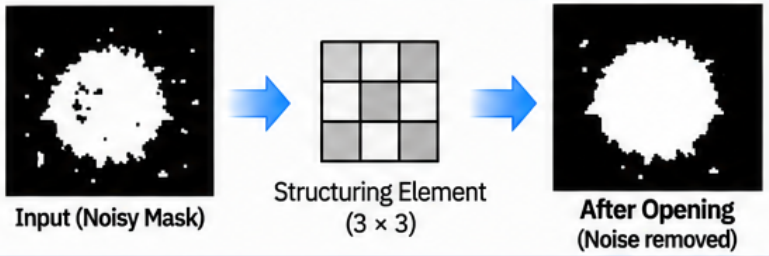
- Contains stray noisy pixels
- Irregular boundaries
- Small holes inside the disc
- Not a clean geometric shape

2. Morphological Operations

(Opening to remove noise, Closing to fill holes)

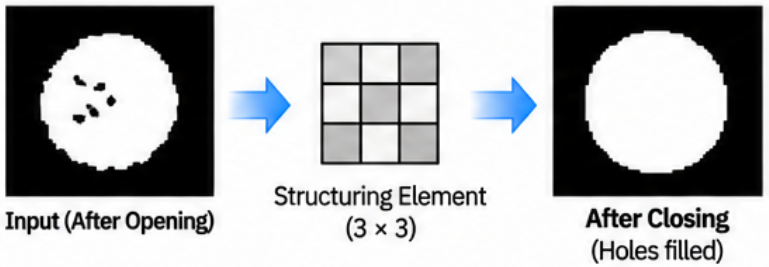
Opening (Erosion → Dilation)

Removes small noisy pixels and stray objects



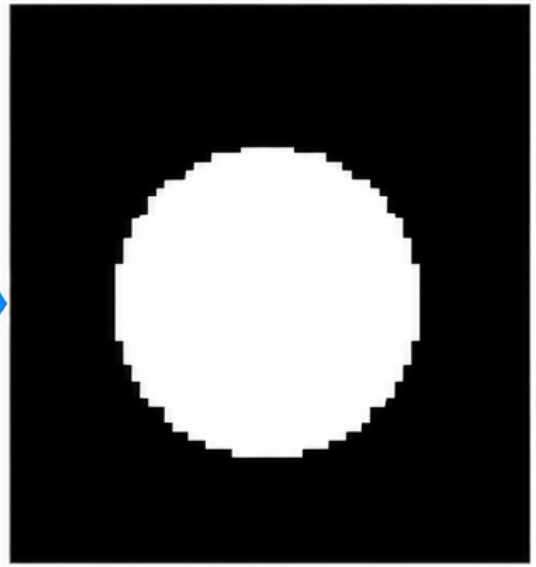
Closing (Dilation → Erosion)

Fills small holes and gaps inside the disc



3. Refined Disc Mask

(Final clean binary mask)



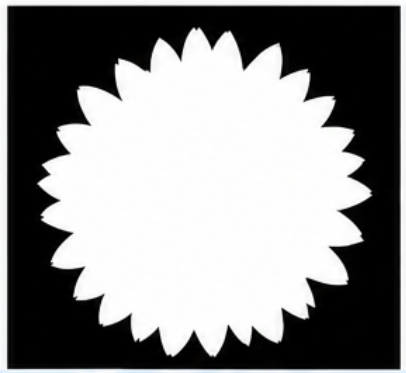
- Noise pixels removed
- Internal holes filled
- Smooth and solid disc region
- Clean geometric shape for feature extraction

How it works:

1. Start with the raw binary disc mask obtained from K-Means, which may contain **stray noise pixels** and **small holes**.
2. Apply **Morphological Opening (Erosion followed by Dilation)** to remove small isolated noise pixels and unwanted artifacts.
3. Apply **Morphological Closing (Dilation followed by Erosion)** to **fill internal holes** and gaps within the disc region.
4. These operations use a small **structuring element** (e.g., 3 × 3) to process the binary mask based on neighborhood pixels.
5. Result: a clean, smooth, and solid binary mask of the central disc, suitable for accurate geometric and feature extraction.

Phase 3H: Shape & Geometry Metrics (Decomposed Proof)

1. Input Binary Masks

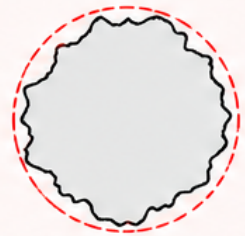


Whole-Flower Mask
(Petals + Disc)

Disc Mask
(Central Pollen Disc)

2. Geometric Feature Calculation (5 Features)

Circularity (How circle-like the shape is)



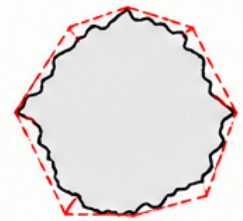
Actual boundary
Equivalent circle (same area)

$$\text{Circularity} = \frac{4\pi A}{P^2}$$

A = Area of shape
P = Perimeter of shape

Value close to 1 indicates a perfect circle

Solidity (How solid the shape is)



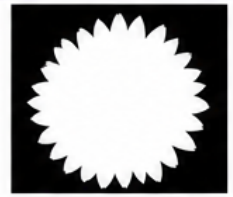
Actual shape
Convex hull (smallest convex polygon)

$$\text{Solidity} = \frac{A}{A_{hull}}$$

A = Area of shape
A_{hull} = Area of convex hull

Value close to 1 indicates a solid shape (without major indentations)

Disc Ratio (Relative size of disc in flower)



Area_{flower} (A_f)

Area_{disc} (A_d)

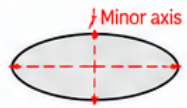
$$\text{Disc Ratio} = \frac{A_d}{A_f}$$

A_d = Area of disc mask
A_f = Area of whole-flower mask

Indicates the proportion of the central disc relative to the entire flower size

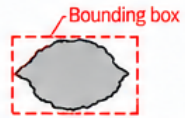
Additional Geometric Metrics (2 more features)

1. Eccentricity (elongation)



$$\text{Eccentricity} = \sqrt{1 - \left(\frac{\text{Minor}}{\text{Major}}\right)^2}$$

2. Extent (how much of bounding box is filled)



$$\text{Extent} = \frac{A}{A_{bbox}}$$

A = Area of shape
A_{bbox} = Area of bounding box

3. Output: 5-D Geometric Feature Vector

5 Numerical Features

1. **Circularity**	0.35
2. **Solidity**	0.88
3. **Disc Ratio**	0.28
4. **Eccentricity**	0.12
5. **Extent**	0.76

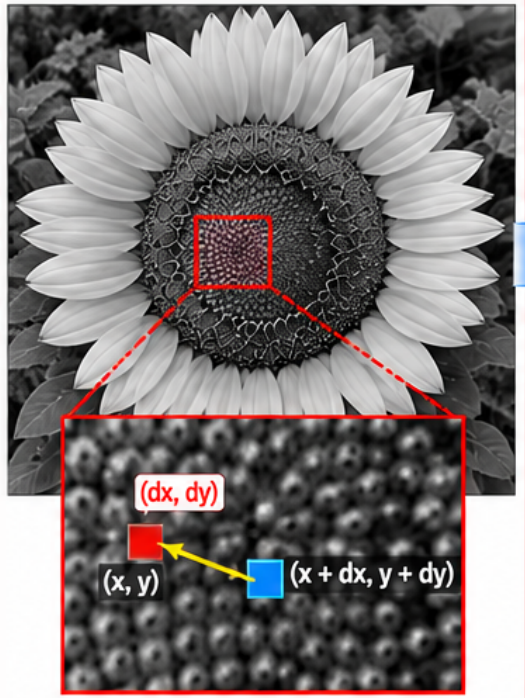
Final 5-D Vector
[0.35, 0.88, 0.28, 0.12, 0.76]

How it works:

1. Use the clean binary masks of the whole flower (petals + disc) and the central disc (from previous steps).
2. Compute geometric properties using shape analysis: Circularity, Solidity, Disc Ratio, Eccentricity, and Extent.
3. These metrics are calculated using area, perimeter, convex hull, and bounding box measurements from the masks.
4. Each metric captures an important geometric characteristic of the sunflower (shape regularity, solidness, relative disc size, etc.).
5. **Result**: the visual masks are converted into a compact 5-dimensional numerical feature vector representing the flower's geometry.

Phase 3I: Texture / GLCM Metrics (Decomposed Proof)

1. Input Grayscale Image with Pixel Pair Analysis



Zoomed Patch with Pixel Pair (dx, dy)
Pixel intensity comparison at a given offset
(e.g., dx = 1, dy = 0)

2. GLCM: Pixel Co-occurrence Matrix

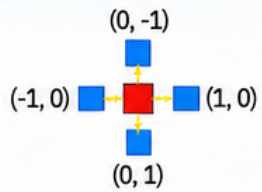
Counts how often pairs of pixel intensities occur

Gray Levels: 0 – 3 (Example 4 × 4 matrix)

i (Reference pixel intensity) \ j (Neighbor pixel intensity)	0	1	2	3
0	12	5	2	1
1	6	18	7	3
2	4	9	20	6
3	1	3	5	14

GLCM(i, j) = number of times a pixel with intensity *i* occurs with a neighboring pixel with intensity *j* at offset (dx, dy)

Common Offsets



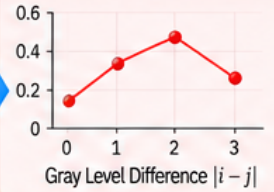
Typically computed for:
- (1, 0) 0° (horizontal)
- (0, 1) 90° (vertical)
- (1, 1) 45° (diagonal)
- (-1, 1) 135° (diagonal)

3. Texture Features from GLCM

Four statistical measures

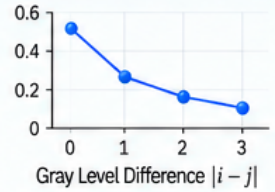
Contrast
(Measures local intensity variation)

$$\text{Contrast} = \sum_{i,j} (i-j)^2 P(i,j)$$



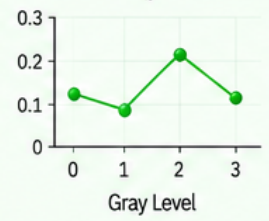
Homogeneity
(Measures closeness of distribution)

$$\text{Homogeneity} = \sum_{i,j} \frac{P(i,j)}{1+|i-j|}$$



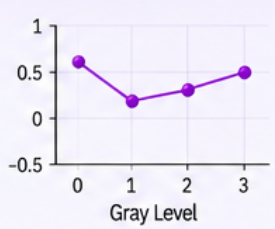
Energy
(Measures uniformity)

$$\text{Energy} = \sum_{i,j} P(i,j)^2$$



Correlation
(Measures linear dependency)

$$\text{Correlation} = \frac{\sum_{i,j}(i-\mu_i)(j-\mu_j)P(i,j)}{\sigma_i \sigma_j}$$



4. Output: 4-D Texture Feature Vector

4 Numerical Features

1. Contrast	0.42
2. Homogeneity	0.81
3. Energy	0.26
4. Correlation	0.67

Final 4-D Vector
[0.42, 0.81, 0.26, 0.67]

How it works:

1. Take the grayscale image (typically the enhanced L-channel) and select a **local patch** from the sunflower disc.
2. For each pixel, count how often a gray level *i* occurs with a neighboring gray level *j* at a specific offset (dx, dy) to build the **Gray Level Co-occurrence Matrix (GLCM)**.
3. The GLCM captures the spatial relationship between pixel intensities, representing surface texture patterns (smooth, rough, repetitive, etc.).
4. Compute four statistical measures from the GLCM: **Contrast** (intensity variation), **Homogeneity** (closeness), **Energy** (uniformity), and **Correlation** (linear dependency).
5. The sunflower disc texture is thus mathematically converted into a **4-dimensional numerical feature vector** for use in the next stage.

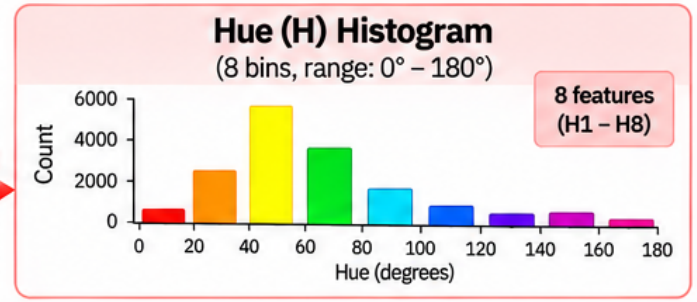
Phase 3J: Color / HSV Histogram Metrics (Decomposed Proof)

1. Input Masked Sunflower Image
(in RGB Color Space)

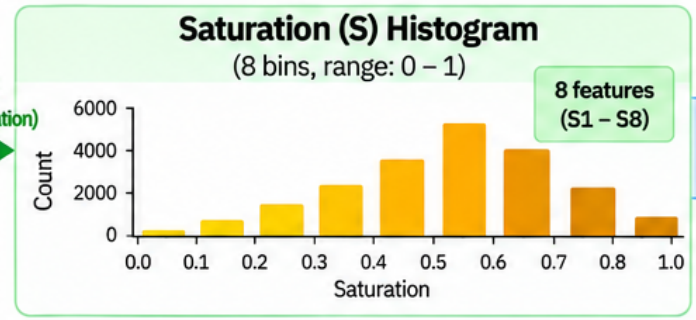
Input image containing only sunflower pixels (petals + disc) using the whole-flower mask

HSV Conversion

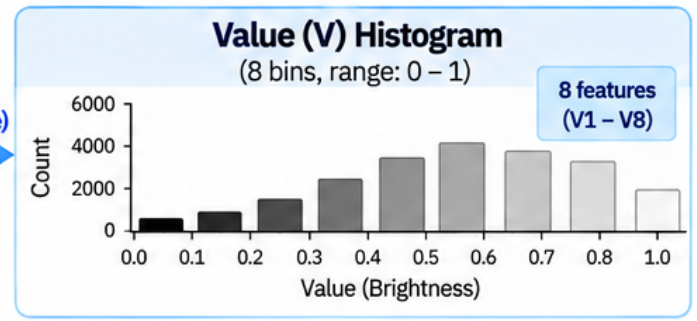
H (Hue)



S (Saturation)



V (Value)



3. Output: 26-D Color Feature Vector
(8 H + 8 S + 8 V + 2 additional features)

Hue Histogram Features (H1 – H8)		
1.	H1	0.02
2.	H2	0.08
3.	H3	0.21
4.	H4	0.18
5.	H5	0.09
6.	H6	0.05
7.	H7	0.03
8.	H8	0.02
Saturation Histogram Features (S1 – S8)		
9.	S1	0.01
10.	S2	0.03
11.	S3	0.07
12.	S4	0.12
13.	S5	0.18
14.	S6	0.22
15.	S7	0.19
16.	S8	0.11
Value Histogram Features (V1 – V8)		
17.	V1	0.02
18.	V2	0.04
19.	V3	0.08
20.	V4	0.15
21.	V5	0.21
22.	V6	0.20
23.	V7	0.18
24.	V8	0.12
Additional Color Features		
25.	Mean Hue	0.26
26.	Mean Saturation	0.58

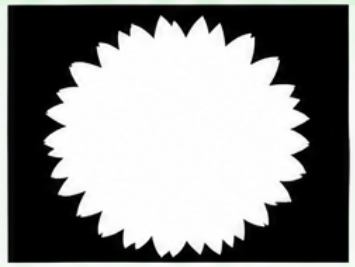
How it works:

1. Use the masked sunflower image (only petals + disc) and convert the RGB pixels to HSV (Hue, Saturation, Value) color space.
2. For each channel (H, S, V), compute an 8-bin normalized histogram to capture the color distribution.
3. Concatenate the 8 Hue, 8 Saturation, and 8 Value histogram values (24 features).
4. Add 2 additional color features (e.g., mean hue and mean saturation) to capture overall color characteristics.
5. **Result:** a 26-dimensional numerical feature vector representing the color information of the sunflower, used for the next stage of analysis.

Phase 4K: Mathematical Image Destruction (Convergence to 35D Vector)

1. Input Sunflower Image

2. Whole-Flower Mask



3. Disc Mask

4. Decomposed Feature Extraction (Total Image Destruction)

Shape & Geometry Metrics
(5 features)

- Circularity
- Solidity
- Disc Ratio
- Eccentricity
- Extent

5 numbers
[0.35, 0.88, 0.28, 0.12, 0.76]

Texture / GLCM Metrics
(4 features)

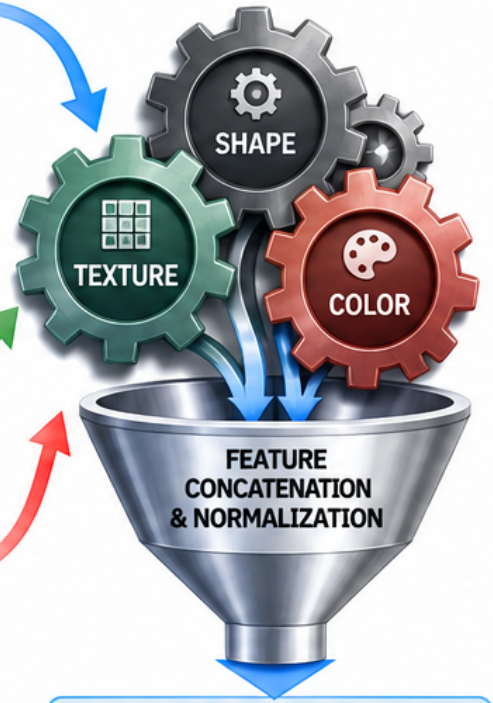
- Contrast
- Homogeneity
- Energy
- Correlation

4 numbers
[0.42, 0.81, 0.26, 0.67]

Color / HSV Histogram Metrics
(26 features)

- 8 Hue histogram bins
- 8 Saturation histogram bins
- 8 Value histogram bins
- 2 additional color features (mean hue, mean saturation)

26 numbers
[H1..H8, S1..S8, V1..V8, meanH, meanS]



Complete Destruction of Visual Information
(Image → Numbers Only)

5. Output
Raw 35D Feature Vector

1	0.35
2	0.88
3	0.28
4	0.12
5	0.76
6	0.42
7	0.81
8	0.26
9	0.67
10	0.02
11	0.08
12	0.21
13	0.18
14	0.09
15	0.05
⋮	⋮
31	0.20
32	0.18
33	0.12
34	0.26
35	0.58

How it works:

1. The input sunflower image and its segmented masks (whole-flower and disc) are processed through mathematical analysis.
2. Shape & geometry metrics (5), texture/GLCM metrics (4), and color/HSV histogram metrics (26) are extracted from the image and masks.
3. These measurements capture all essential visual information (shape, texture, color) in numerical form.
4. The features are concatenated and normalized, completely **destroying** the original visual structure of the image.
5. **Result:** a single 35-dimensional numerical feature vector that fully represents the sunflower, replacing the entire image with 35 numbers.

Features of the obtained numeric dataset from raw jpeg image

What are the 4 GLCM Features? (Located at the very end of the row)
GLCM (Gray-Level Co-occurrence Matrix) evaluates the **texture** of the flower. It looks at the grayscale image and calculates how often pairs of pixels with specific brightness values appear next to each other. We extract exactly 4 numbers from this matrix:

1. **Contrast:** Measures the local intensity variation. A wilted, shriveled sunflower with harsh shadows on its petals will have a very **high contrast**. A smooth, fresh young bud will have low contrast.

2. **Homogeneity:** Measures how uniform the texture is. If the flower petals are all a smooth, consistent yellow without blemishes, homogeneity will be **high**.

3. **Energy:** Measures the "orderliness" of the image. Highly predictable, repeating patterns (like the tightly packed seeds in a healthy central disc) result in higher energy.

4. **Correlation:** Measures how strongly a pixel is mathematically related to its immediate neighbor. It helps the model understand the directional grain of the flower petals.

Features of the obtained numeric dataset from raw jpeg image

What are the other 31 features in the row?
Before the 4 GLCM features at the end of the row, there are 31 other numeric features that describe the shape and color of the flower:

Geometric Shape (5 features):

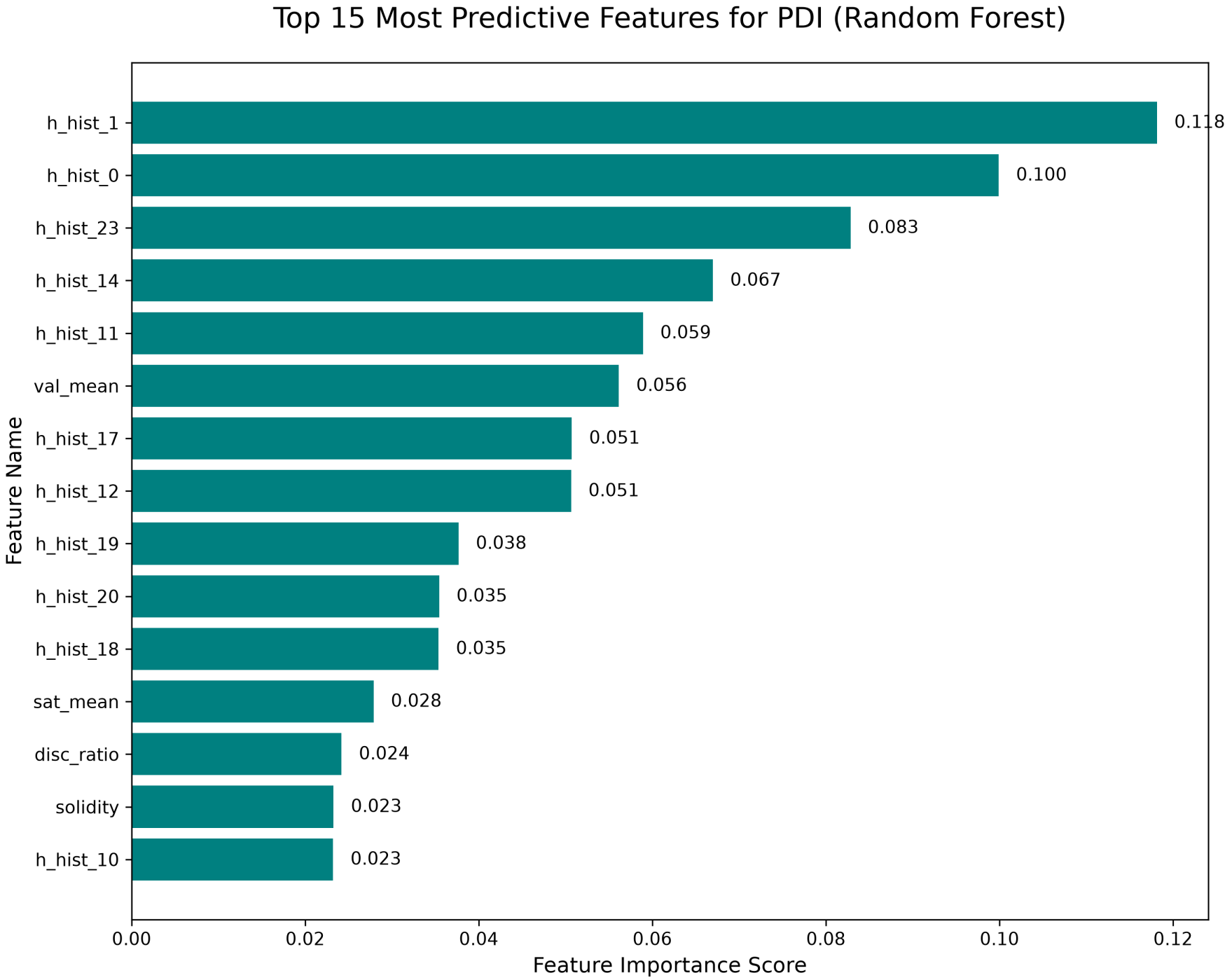
- **area_ratio:** How much of the image is taken up by the flower vs. the background.
- **extent:** The ratio of the flower's area to its bounding box.
- **solidity:** How "solid" the flower mask is (does it have missing petals/gaps?).
- **circularity:** How perfectly round the flower is.
- **disc_ratio: (Crucial for PDI)** The size of the central pollen disc compared to the whole flower.

Color Brightness (2 features):

- **sat_mean:** The average color saturation (vividness) of the central disc.
- **val_mean:** The average brightness of the central disc.

Color Histograms (24 features):

- **h_hist_0 through h_hist_23:** These are the 24 columns in the middle of your CSV. They represent the HSV color histogram. We take the Hue, Saturation, and Value channels and divide each into 8 buckets (8 bins × 3 channels = 24 features). This tells the Machine Learning model exactly how much yellow, green, and brown is present in the flower!


Top 15 Most Predictive Features for PDI (Random Forest)
Feature Name
h_hist_1
h_hist_0
h_hist_23
h_hist_14
h_hist_11
val_mean
h_hist_17
h_hist_12
h_hist_19
h_hist_20
h_hist_18
sat_mean
disc_ratio
solidity
h_hist_10
0.118
0.100
0.083
0.067
0.059
0.056
0.051
0.051
0.038
0.035
0.035
0.028
0.024
0.023
0.023
0.00
0.02
0.04
0.06
0.08
0.10
0.12
Feature Importance Score

Random Forest Classification

35 Features
(Image Data)

Feature 1
Feature 2
Feature 3
⋮
Feature 35

Bootstrap Sampling

Decision Tree 1 → Prediction: **Low PDI**

Decision Tree 2 → Prediction: **Medium PDI**

Decision Tree 3 → Prediction: **Low PDI**

Decision Tree 4 → Prediction: **High PDI**

Decision Tree 5 → Prediction: **Low PDI**

Majority Vote

Final Prediction

- **Low PDI**
- **Medium PDI**
- **High PDI**

In the Classical Phase: Gini Impurity (Random Forest)

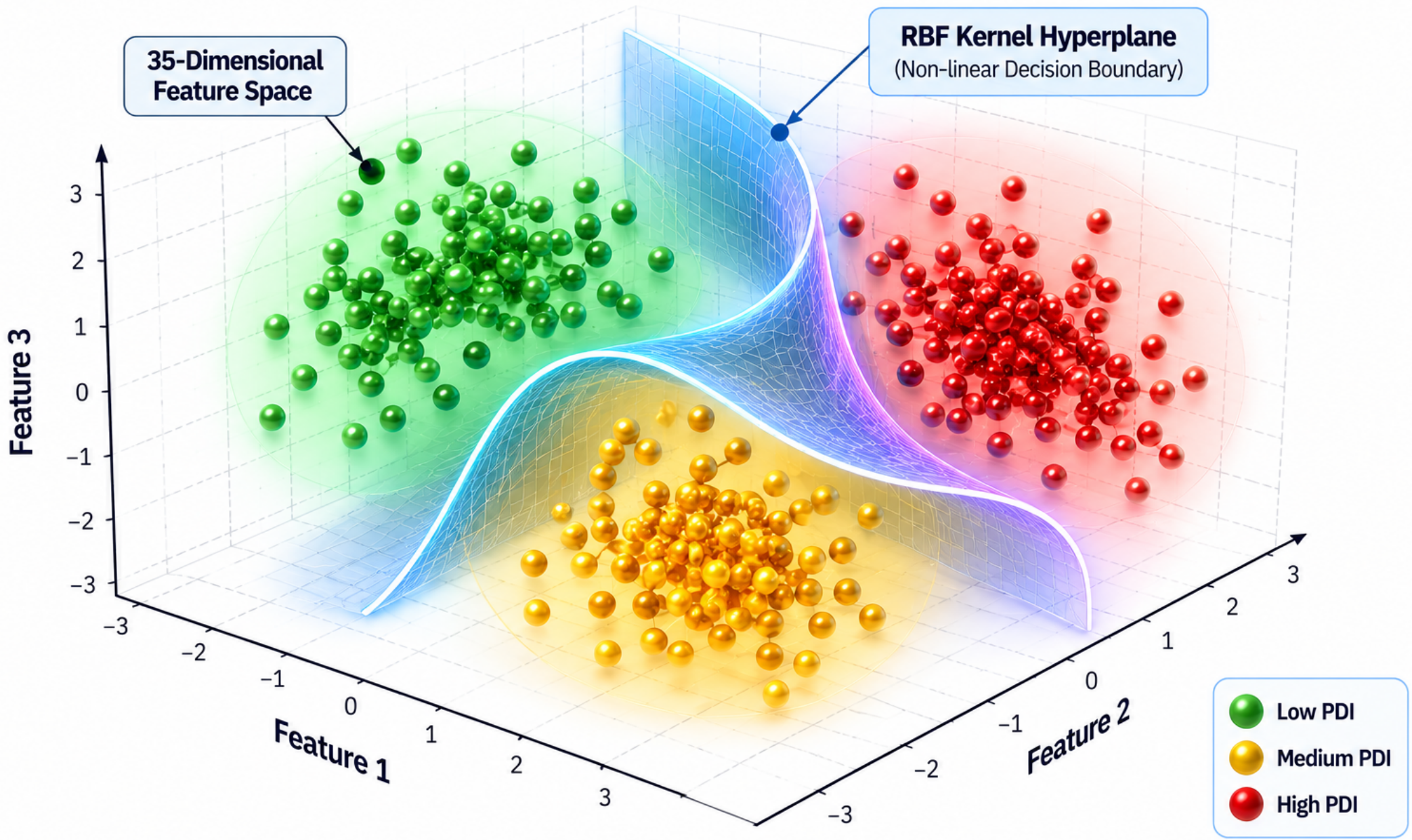
For the Classical pipeline, we don't pick the features ourselves—the Random Forest calculates them automatically using a mathematical concept called **Gini Impurity** (or Mean Decrease in Impurity).

- **How it works:** Imagine a tree has a messy bucket of 1,000 mixed flowers (Low, Medium, and High). The tree tests a rule: "*Split them based on the Area Ratio.*" It checks the two new buckets. Are they perfectly sorted? If not, the tree tries another rule: "*Split them based on Disc Brightness.”*

- **The Math:** Every time a feature successfully splits a messy bucket into two "pure" buckets (e.g., all the Lows end up on one side, all the Highs on the other), that feature is awarded points for reducing “impurity."

- **The Result:** The algorithm adds up the points for all 35 features across all 300 trees. The features that consistently created the cleanest splits (like the Color Histograms and Disc Brightness) are mathematically ranked as the "Most Important."

Support Vector Machine (SVM) with RBF Kernel



To classify the flowers, we used a Support Vector Machine (SVM) equipped with an RBF Kernel.

- First, the SVM takes the **35 numerical features** we extracted from each image and plots them as data points in space. Because our data is complex and overlapping, you cannot simply draw a straight line to separate the Low, Medium, and High PDI flowers.

- To solve this, the model uses the **RBF (Radial Basis Function) Kernel**. This function acts as a mathematical 'similarity ruler.' It measures the radial distance between the data points, which mathematically projects the messy data into a much higher-dimensional space where the clusters naturally untangle.

- Once untangled, the SVM calculates the absolute **widest possible boundary** between the classes to ensure it doesn't just memorize the data, but generalizes well to new images. It anchors these boundary walls using only the most critical, edge-case data points, known as **Support Vectors**.

- Ultimately, this allows the SVM to draw flexible, highly-accurate boundaries around our PDI clusters, resulting in the **~89% classical accuracy** you see in our results.

Phase 4L: Data Compression (PCA 35D to 6D)

1. Input
Raw 35D Feature Vector

Index	Value
1	0.35
2	0.88
3	0.28
4	0.12
5	0.76
6	0.42
7	0.81
8	0.26
9	0.67
10	0.02
⋮	⋮
31	0.20
32	0.18
33	0.12
34	0.26
35	0.58

2. Principal Component Analysis (PCA)
Linear dimensionality reduction

PCA

Standardize features
(mean = 0, variance = 1)

Compute covariance matrix Σ

$$\Sigma = \frac{1}{n-1} X^T X$$

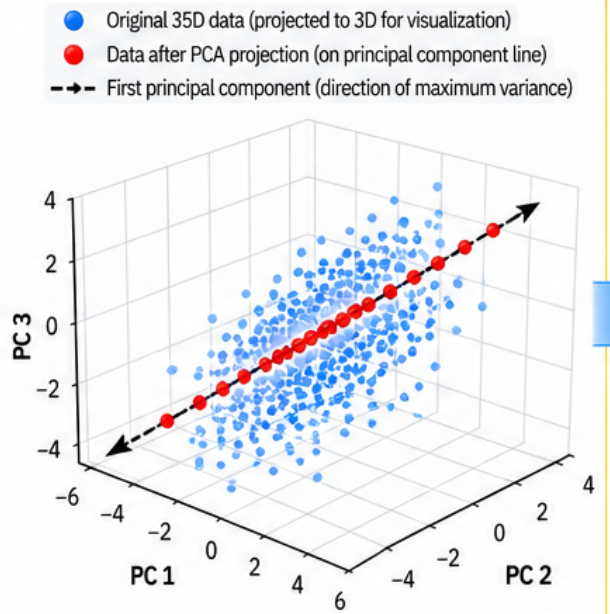
Eigen decomposition

$$\Sigma = V \Lambda V^T$$

Project onto top 6 principal components

$$Z = XW_6$$

3. 3D Visualization of PCA Projection
Data projected from 35D to 6D (shown in 3D for illustration)



PCA finds the directions (principal components) that capture the **maximum variance** in the data. The data is projected onto the **top 6 components**, removing **redundant information and noise.**

4. Output
Compressed 6D Feature Vector
(Compact and optimized)

45.2	-12.8	-5.6	8.1	-3.4	1.7

6 Features
(PCA Components 1–6)

Properties:

- Reduced from **35 dimensions** to **6 dimensions** (≈ **83% compression**)
- Captures **maximum variance** (information)
- Removes **redundant and correlated** features
- Reduces **noise**
- Retains the **most important information** for analysis and modeling

How it works:

1. The **35-dimensional** feature vector is first **standardized** so that all features have zero mean and unit variance.
2. PCA computes the **covariance matrix** to analyze the relationships between features and performs **eigen decomposition** to find the principal components.
3. The data is projected onto the **top 6 principal components**, which capture the **maximum variance** in the original data.
4. This process **removes redundant, correlated, and noisy information**, keeping only the **most important** patterns.
5. **Result:** a compact 6-dimensional feature vector that preserves the essential information of the original 35 features, optimized for better performance in the next stage (e.g., classification or clustering).

In the Quantum Phase: Principal Component Analysis (PCA)

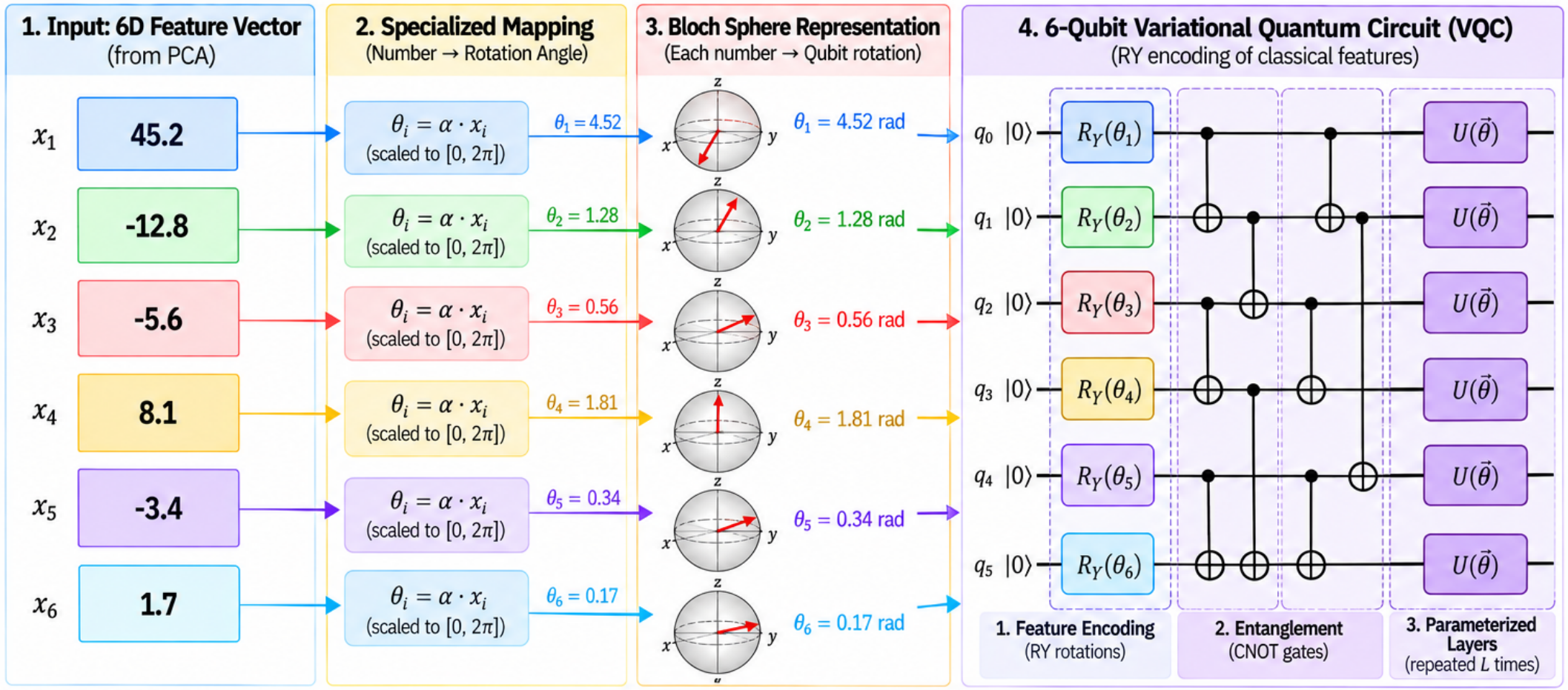
In Phase 2, we can't fit 35 features onto a 6-qubit quantum computer. We have to reduce the data to the 6 most important features. But we don't just throw away 29 features! We use **PCA (Principal Component Analysis)**.

- **How it works:** PCA doesn't just "pick" the top 6 features. Instead, it looks at the entire 35-dimensional dataset and mathematically crushes it down.

- **The Math (Variance):** PCA calculates the *Variance* (how spread out the data is). It looks for the mathematical direction where the flowers differ the most from each other, and calls that "Component 1". Then it finds the next biggest difference, and calls that "Component 2", and so on.

- **The Result:** PCA mathematically blends our 35 original features into **6 brand-new super-features** (Principal Components). These 6 numbers contain over 80% of the information from the original 35 features, which is why our 6-qubit Quantum Circuit was still able to achieve 97% accuracy!

Phase 4M: Handoff to Quantum: VQC Encoding



How it works:

1. The 6-dimensional feature vector (from PCA) is taken as input.
2. Each numerical value x_i is scaled and mapped to a rotation angle θ_i using a linear transformation (e.g., $\theta_i = \alpha \cdot x_i$, scaled to $[0, 2\pi]$).
3. Each angle θ_i is represented on a Bloch sphere as a rotation around the Y-axis ($R_Y(\theta_i)$), encoding the classical information into the quantum state of qubit q_i.
4. The 6 qubits are then entangled using CNOT gates, allowing the model to capture complex relationships between features.
5. The encoded and entangled quantum state is passed through parameterized variational layers $U(\vec{\theta})$ (repeated L times) for training.
6. **Result:** the classical 6 numerical features are completely converted into abstract quantum parameters, forming the input to the Variational Quantum Circuit (VQC).

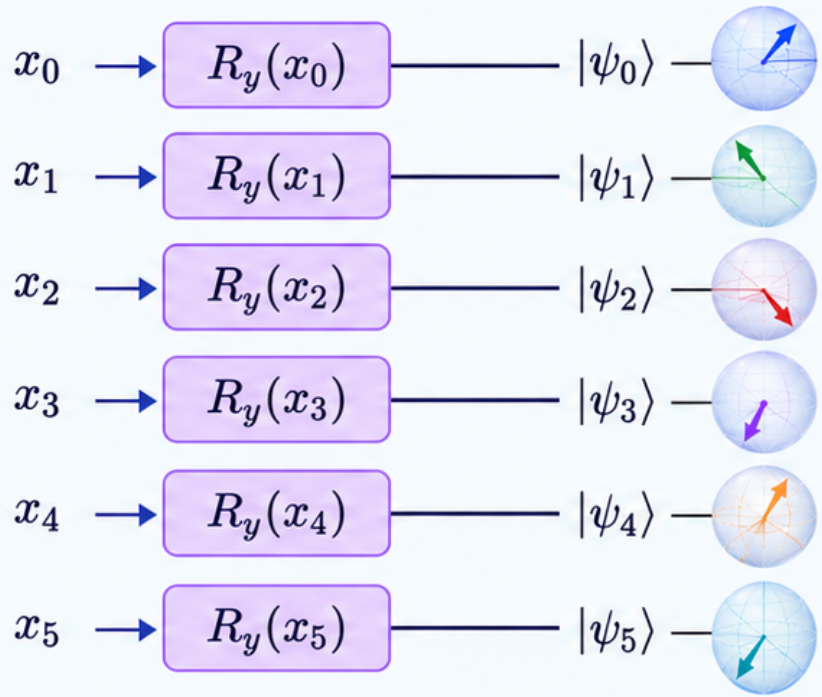
1 Step 1: Angle Encoding (Loading Classical Data into Qubits)

PCA Output
(6 Numerical Features)

x_0	0.25
x_1	-1.12
x_2	0.78
x_3	-0.36
x_4	1.03
x_5	-0.91

PCA
(6 Features)

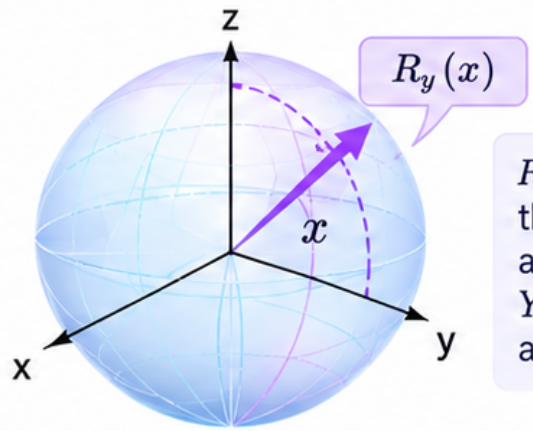
Angle Encoding with R_y Gates
Load each feature x_i into qubit i using a Y-rotation gate



Y-Rotation Gate (R_y)

$$R_y(x) = \begin{pmatrix} \cos\left(\frac{x}{2}\right) & -\sin\left(\frac{x}{2}\right) \\ \sin\left(\frac{x}{2}\right) & \cos\left(\frac{x}{2}\right) \end{pmatrix}$$

Rotation on Bloch Sphere



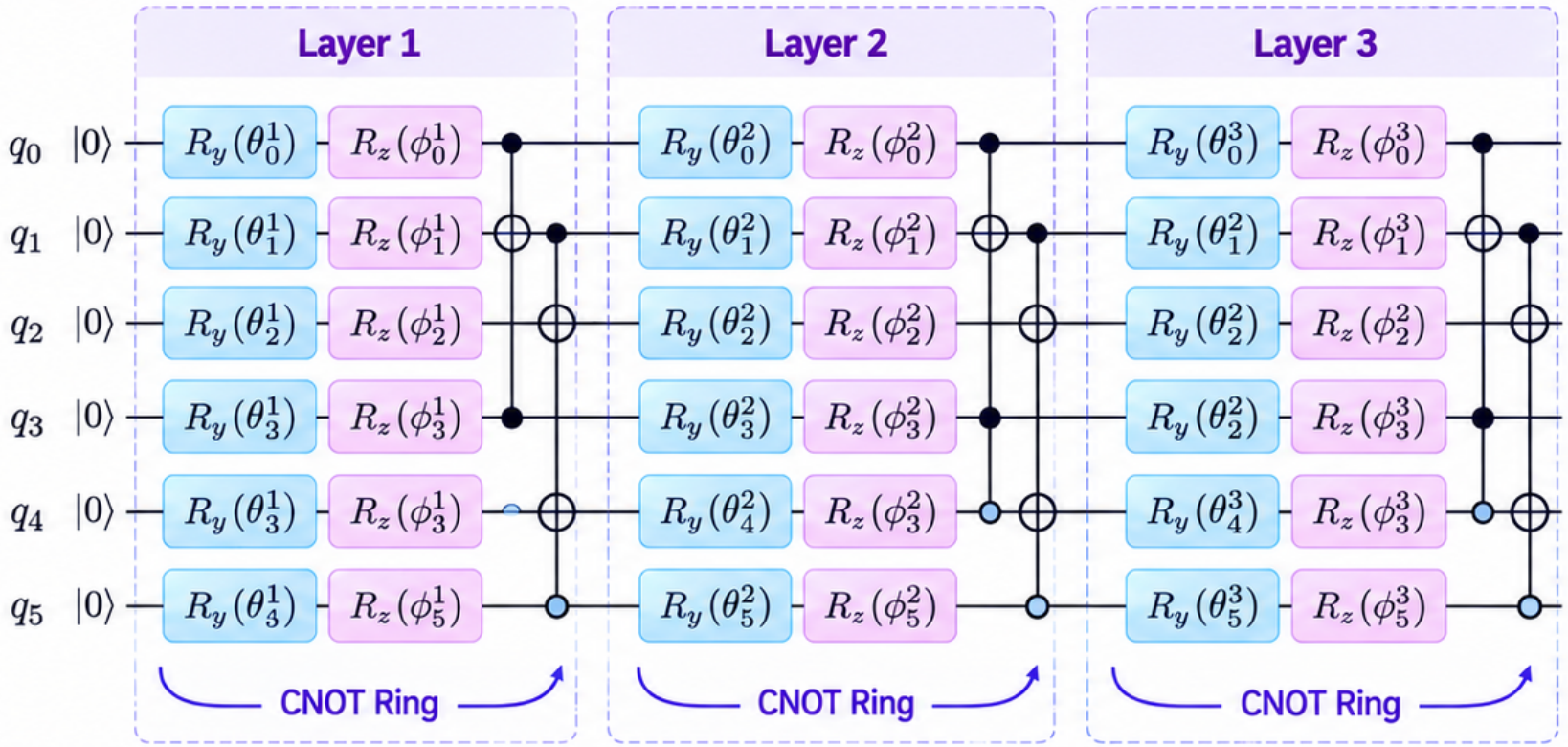
$R_y(x)$ rotates the qubit's state around the Y-axis by an angle x.

Why this works?

Angle encoding transforms the classical numerical features into quantum states (superposition) that can be processed by the quantum circuit.

2 Step 2: Variational Quantum Circuit (VQC)

3 Layers of Trainable Gates + Entanglement (CNOT Ring)



Trainable Rotation Gates

Y-Rotation

$$R_y(\theta_i)$$

θ_i : trainable parameter

$$R_y(\theta) = \begin{pmatrix} \cos\left(\frac{\theta}{2}\right) & -\sin\left(\frac{\theta}{2}\right) \\ \sin\left(\frac{\theta}{2}\right) & \cos\left(\frac{\theta}{2}\right) \end{pmatrix}$$

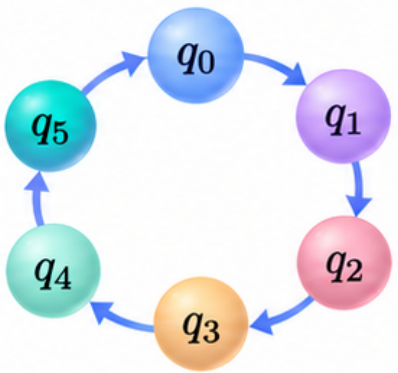
Z-Rotation

$$R_z(\phi_i)$$

ϕ_i : trainable parameter

$$R_z(\phi) = \begin{pmatrix} e^{-i\phi/2} & 0 \\ 0 & e^{i\phi/2} \end{pmatrix}$$

CNOT Ring Entanglement



Ring Connections:

$q_0 \rightarrow q_1$
$q_1 \rightarrow q_2$
$q_2 \rightarrow q_3$
$q_3 \rightarrow q_4$
$q_4 \rightarrow q_5$
$q_5 \rightarrow q_0$

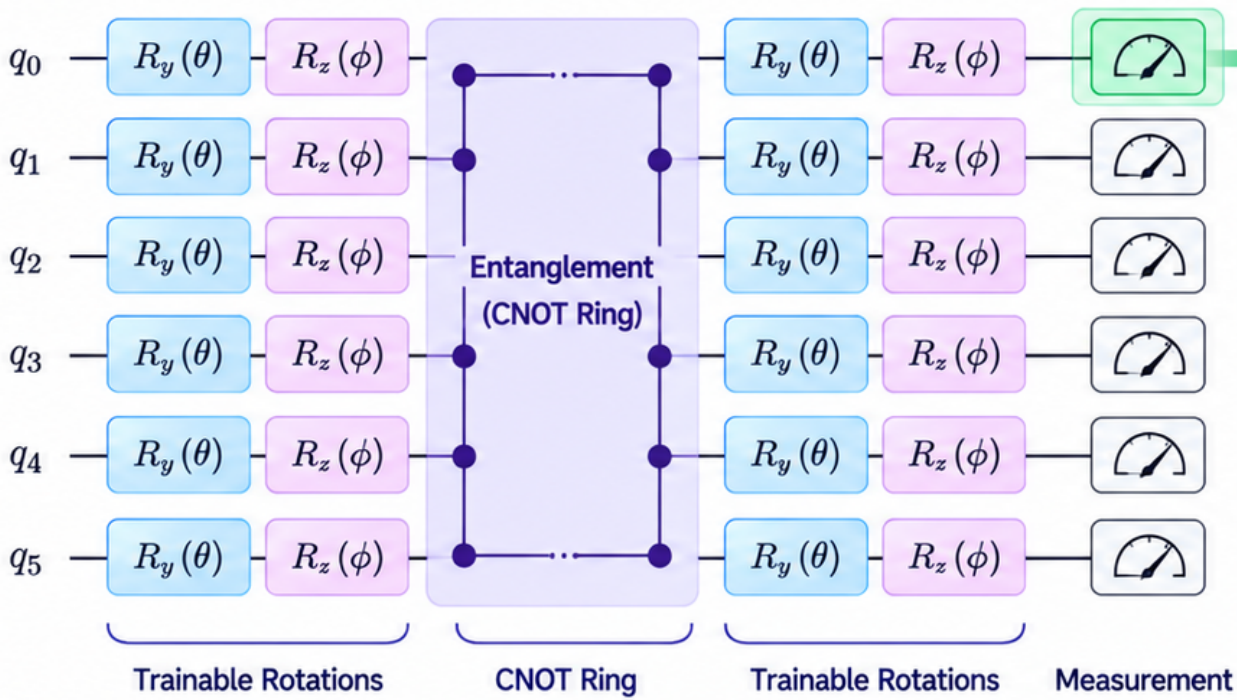
Key Idea

Each layer applies trainable rotations $R_y(\theta)$ and $R_z(\phi)$ to every qubit, followed by a CNOT ring to entangle the qubits. This allows the quantum state of one feature to influence the state of another, enabling powerful multi-feature learning.

3 Step 3: Measurement (The Output)

Measure the expectation value of the Pauli-Z operator on the first qubit (q_0)

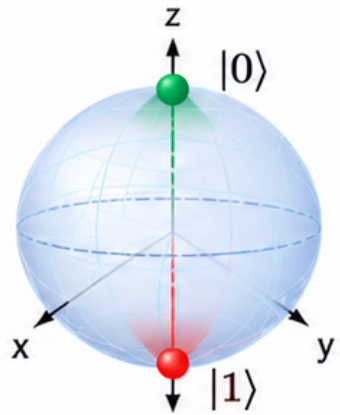
Quantum Circuit (After VQC)



Expectation Value

$$\langle Z_0 \rangle$$

Pauli-Z Measurement



Pauli-Z Operator

$$Z = \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix}$$

Expectation Value

$$\langle Z_0 \rangle = \langle \psi | Z \otimes I \otimes \ldots | \psi \rangle$$

$$-1 \leq \langle Z_0 \rangle \leq +1$$

Interpretation of $\langle Z_0 \rangle$

$\langle Z_0 \rangle \approx -1$

Stronger $|1\rangle$ measurement tendency

$\langle Z_0 \rangle \approx +1$

Stronger $|0\rangle$ measurement tendency

Key Idea

We measure the expectation value of the Pauli-Z operator on the first qubit q_0, which gives a value between -1 and $+1$, indicating the qubit's tendency towards the $|0\rangle$ or $|1\rangle$ state.

4 Step 4: Classical Hybrid Head (Sigmoid Formula)

Convert the quantum measurement to a probability for binary classification

Input from Quantum Circuit

Quantum Measurement Output

$$\langle Z_0 \rangle$$

$$-1 \leq \langle Z_0 \rangle \leq +1$$

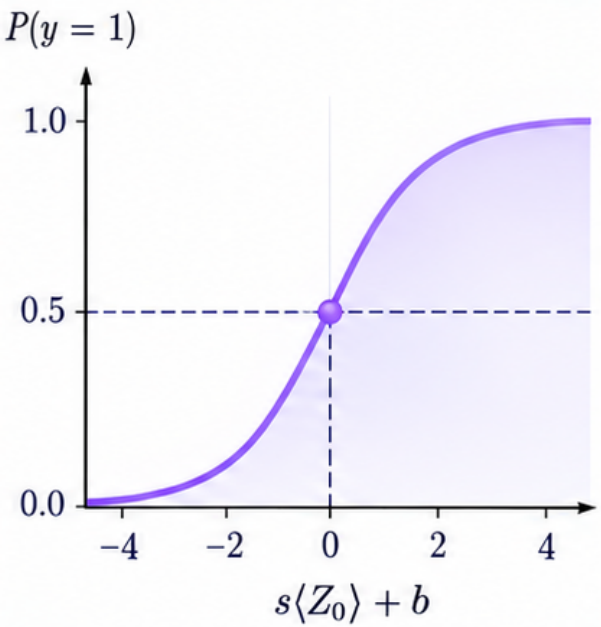
Classical Sigmoid Function

Convert $\langle Z_0 \rangle$ to a probability between 0 and 1

$$P(y=1) = \frac{1}{1 + \exp\left[-\left(s\langle Z_0 \rangle + b\right)\right]}$$

$\langle Z_0 \rangle$	=	quantum measurement output (−1 to +1)
s	=	trainable scale parameter
b	=	trainable bias parameter
$P(y=1)$	=	predicted probability of the positive class (High PDI)

Sigmoid Function Curve



Final Decision

$P(y=1) < 0.5$

Low PDI

$P(y=1) \geq 0.5$

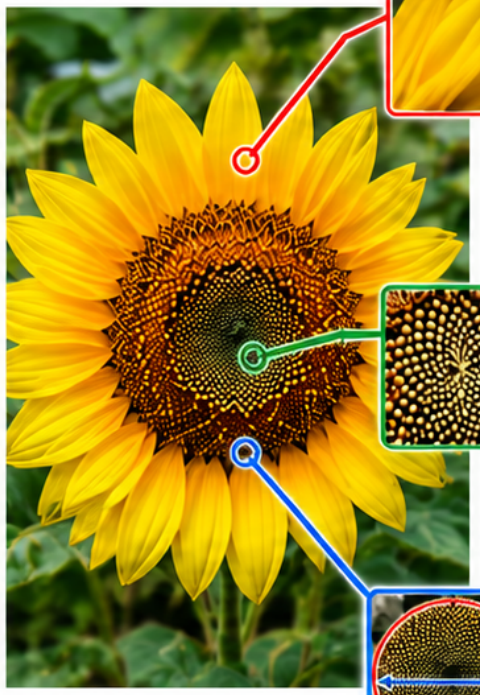
High PDI

Key Idea

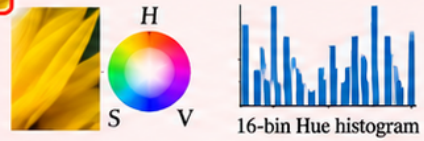
The quantum measurement $\langle Z_0 \rangle$ (ranging from -1 to $+1$) is passed through a trainable sigmoid function to obtain a probability between 0% and 100% for binary classification (Low PDI or High PDI).

Quantum Method 1: PCA + Angle Encoding (Dimension Reduction)

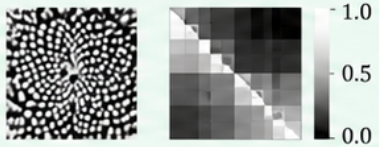
1. Classical Feature Extraction (Computer Vision)



Color Domain
HSV Saturation moments
& 16-bin Hue histogram



Texture Domain
GLCM Contrast,
Homogeneity, Energy



Morphology
Disc Area Ratio
and Circularity

$$Circularity = \frac{4\pi A}{P^2}$$

$$\begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \\ x_5 \\ \vdots \\ x_{31} \end{bmatrix}$$

31-Dimensional
Feature Vector
$x \in \mathbb{R}^{31}$

2. Principal Component Analysis (PCA)

Dimensionality Reduction (31D → 6D)

PCA Transformation

$$x_{6D} = (x_{31D} - \mu) \cdot W$$

x_{31D} : 31-dimensional feature vector
μ : mean vector (31 × 1)
W : orthogonal eigenvector matrix (31 × 6)
x_{6D} : 6-dimensional principal component vector

Eigenvector Matrix W

$$W \in \mathbb{R}^{31\times 6} = \begin{bmatrix} w_{11} & w_{12} & \cdots & w_{16} \\ w_{21} & w_{22} & \cdots & w_{26} \\ w_{31} & w_{32} & \cdots & w_{36} \\ \vdots & \vdots & \ddots & \vdots \\ w_{31,1} & w_{31,2} & \cdots & w_{31,6} \end{bmatrix}$$

Columns of W are the top 6 eigenvectors corresponding to the largest eigenvalues.

3. Reduced Feature Vector

6 Principal Components

$$\begin{bmatrix} z_1 \\ z_2 \\ z_3 \\ z_4 \\ z_5 \\ z_6 \end{bmatrix} \quad \begin{matrix} \text{PC1} \\ \text{PC2} \\ \text{PC3} \\ \text{PC4} \\ \text{PC5} \\ \text{PC6} \end{matrix}$$

6-Dimensional
Principal Component Vector
$x_{6D} \in \mathbb{R}^6$

Cumulative Variance Retained:
86.27%
(Information Loss: 13.73%)

1. PCA Components (Input Features)

PC1	x_1
PC2	x_2
PC3	x_3
PC4	x_4
PC5	x_5
PC6	x_6

$x \in \mathbb{R}^6$

(PCA output)

2. Angle Scaling (Preprocessing)

Scaling Formula

$$\theta_i = \pi \cdot \text{clip}\left(\frac{x_i}{x_{\max}}, -1, 1\right)$$

x_i : i-th PCA component

$x_{\max}$: maximum absolute value (for normalization)

θ_i : rotation angle for qubit i

$\theta_0 \in [-\pi, +\pi]$

$\theta_1 \in [-\pi, +\pi]$

$\theta_2 \in [-\pi, +\pi]$

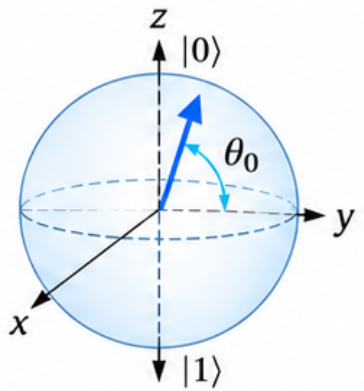
$\theta_3 \in [-\pi, +\pi]$

$\theta_4 \in [-\pi, +\pi]$

$\theta_5 \in [-\pi, +\pi]$

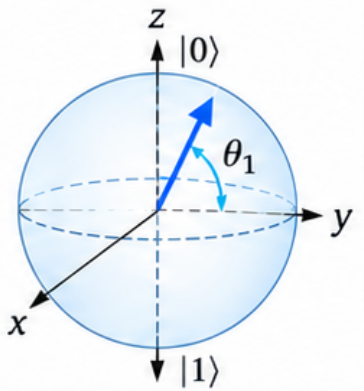
3. Quantum Angle Encoding (RY Rotation on Each Qubit)

Qubit 0 (θ_0)



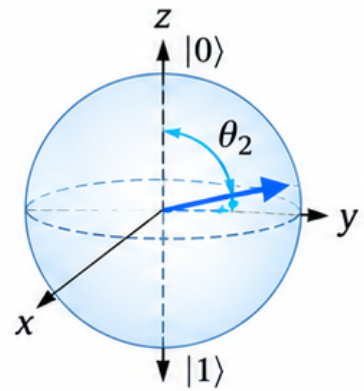
$R_Y(\theta_0)\,|0\rangle = \cos\frac{\theta_0}{2}\,|0\rangle + \sin\frac{\theta_0}{2}\,|1\rangle$

Qubit 1 (θ_1)



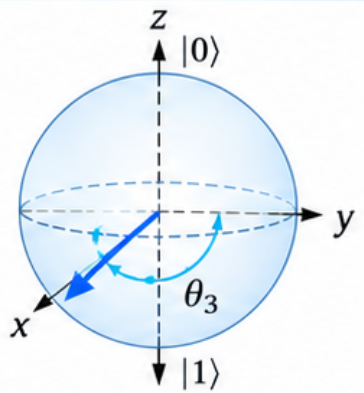
$R_Y(\theta_1)\,|0\rangle = \cos\frac{\theta_1}{2}\,|0\rangle + \sin\frac{\theta_1}{2}\,|1\rangle$

Qubit 2 (θ_2)



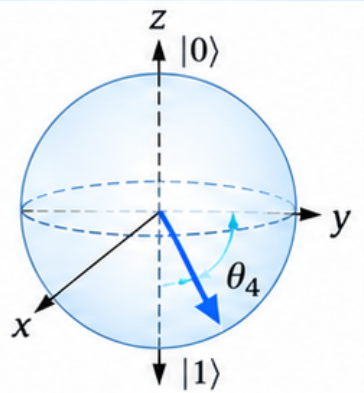
$R_Y(\theta_2)\,|0\rangle = \cos\frac{\theta_2}{2}\,|0\rangle + \sin\frac{\theta_2}{2}\,|1\rangle$

Qubit 3 (θ_3)



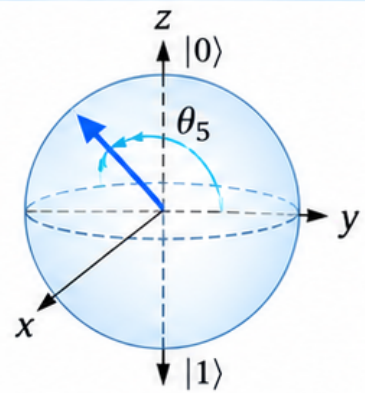
$R_Y(\theta_3)\,|0\rangle = \cos\frac{\theta_3}{2}\,|0\rangle + \sin\frac{\theta_3}{2}\,|1\rangle$

Qubit 4 (θ_4)

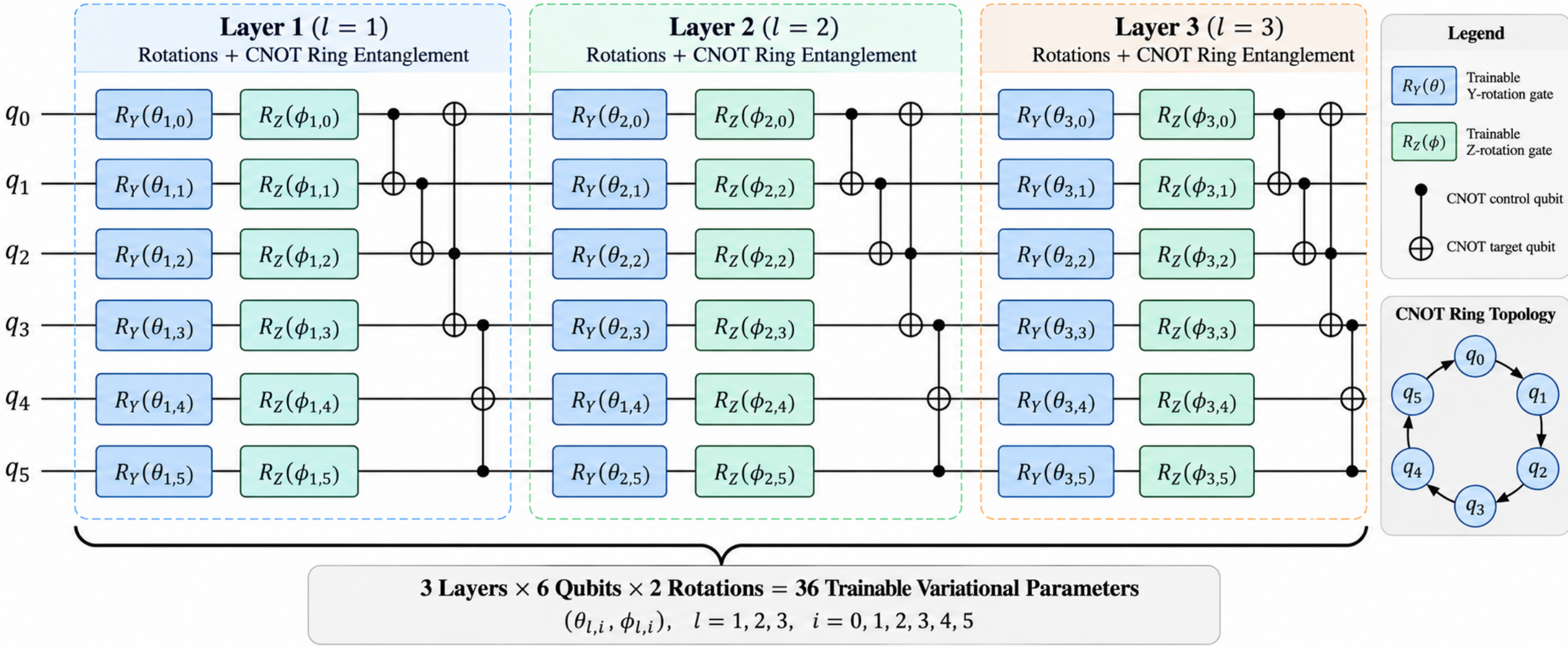


$R_Y(\theta_4)\,|0\rangle = \cos\frac{\theta_4}{2}\,|0\rangle + \sin\frac{\theta_4}{2}\,|1\rangle$

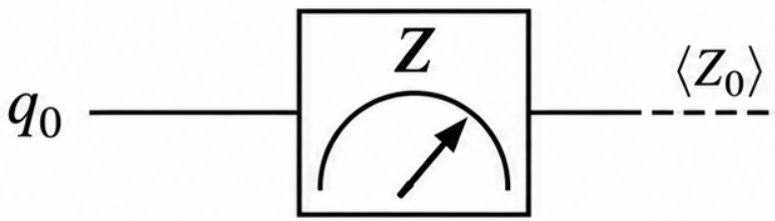
Qubit 5 (θ_5)



$R_Y(\theta_5)\,|0\rangle = \cos\frac{\theta_5}{2}\,|0\rangle + \sin\frac{\theta_5}{2}\,|1\rangle$


Layer 1 ($l = 1$)
Rotations + CNOT Ring Entanglement
Layer 2 ($l = 2$)
Rotations + CNOT Ring Entanglement
Layer 3 ($l = 3$)
Rotations + CNOT Ring Entanglement
q_0
q_1
q_2
q_3
q_4
q_5
$R_Y(\theta_{1,0})$ $R_Z(\phi_{1,0})$ $R_Y(\theta_{2,0})$ $R_Z(\phi_{2,0})$ $R_Y(\theta_{3,0})$ $R_Z(\phi_{3,0})$
$R_Y(\theta_{1,1})$ $R_Z(\phi_{1,1})$ $R_Y(\theta_{2,1})$ $R_Z(\phi_{2,2})$ $R_Y(\theta_{3,1})$ $R_Z(\phi_{3,1})$
$R_Y(\theta_{1,2})$ $R_Z(\phi_{1,2})$ $R_Y(\theta_{2,2})$ $R_Z(\phi_{2,2})$ $R_Y(\theta_{3,2})$ $R_Z(\phi_{3,2})$
$R_Y(\theta_{1,3})$ $R_Z(\phi_{1,3})$ $R_Y(\theta_{2,3})$ $R_Z(\phi_{2,3})$ $R_Y(\theta_{3,3})$ $R_Z(\phi_{3,3})$
$R_Y(\theta_{1,4})$ $R_Z(\phi_{1,4})$ $R_Y(\theta_{1,4})$ $R_Z(\phi_{2,4})$ $R_Y(\theta_{3,4})$ $R_Z(\phi_{3,4})$
$R_Y(\theta_{1,5})$ $R_Z(\phi_{1,5})$ $R_Y(\theta_{2,5})$ $R_Z(\phi_{2,5})$ $R_Y(\theta_{3,5})$ $R_Z(\phi_{3,5})$
Legend
$R_Y(\theta)$ Trainable Y-rotation gate
$R_Z(\phi)$ Trainable Z-rotation gate
CNOT control qubit
CNOT target qubit
CNOT Ring Topology
q_0 q_1 q_2 q_3 q_4 q_5
3 Layers × 6 Qubits × 2 Rotations = 36 Trainable Variational Parameters
$(\theta_{l,i}, \phi_{l,i})$, $l = 1, 2, 3$, $i = 0, 1, 2, 3, 4, 5$

1. Quantum Readout (Pauli-Z Measurement)



Measure expectation value of Pauli-Z on qubit q_0:

$$\langle Z_0 \rangle = \langle \psi | Z_0 | \psi \rangle$$

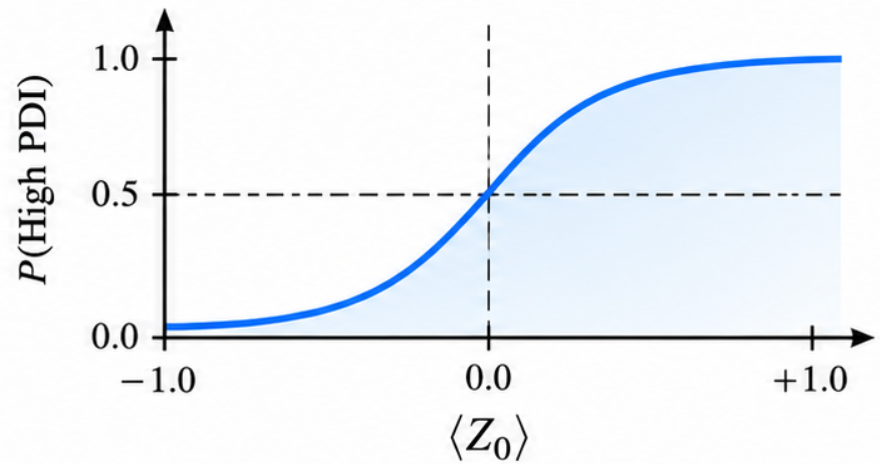
$$\langle Z_0 \rangle \in [-1.0, +1.0]$$

- $\langle Z_0 \rangle \approx +1$ (strong $|0\rangle$ tendency)
- $\langle Z_0 \rangle \approx -1$ (strong $|1\rangle$ tendency)

2. Classical Sigmoid Activation (Probability Mapping)

$$P(\text{High PDI}) = \frac{1}{1 + \exp(-(s \cdot \langle Z_0 \rangle + b))}$$

s = scale parameter b = bias parameter



$P(\text{High PDI})$

3. Classification Decision (Threshold at 0.5)

$P \geq 0.5$
High PDI
(Pollen Deficient)

Threshold: $P = 0.5$

$P < 0.5$
Low PDI
(Pollen Adequate)

Quantum Method 2: Amplitude Encoding (No Dimension Reduction)

1. Raw Feature Vector from Sunflower Imagery
(31 Botanical and Texture Features)

$$x \in \mathbb{R}^{31} = \begin{bmatrix} x_0 \\ x_1 \\ x_2 \\ x_3 \\ \vdots \\ x_{28} \\ x_{29} \\ x_{30} \end{bmatrix}$$

Feature Examples (x_i)

Color Features
- HSV moments
- 16-bin hue histogram
- Color ratios

(24 features)

Texture Features
- GLCM contrast
- GLCM homogeneity
- GLCM energy
- Texture statistics

(4 features)

Morphology Features
- Disc area ratio
- Circularity
- Shape descriptors

(3 features)

$$x = [x_0, x_1, x_2, \ldots, x_{30}]^T \in \mathbb{R}^{31}$$
(31 extracted numerical features)

2. Zero-Padding to 64 Dimensions
(Match $2^6 = 64$ Computational Basis States)

$$x_{\text{pad}} \in \mathbb{R}^{64} = \begin{bmatrix} x_0 \\ x_1 \\ x_2 \\ x_3 \\ \vdots \\ x_{28} \\ x_{29} \\ x_{30} \\ 0 \\ 0 \\ 0 \\ \vdots \\ 0 \end{bmatrix}$$

Original 31 features
$x_0, x_1, \ldots, x_{30}$
(31 dimensions)

Zero padding
$0, 0, \ldots, 0$
(33 dimensions)

$$x_{\text{pad}} = [x_0, x_1, \ldots, x_{30}, 0, 0, \ldots, 0]^T \in \mathbb{R}^{64}$$
(31 original values + 33 zeros = 64 dimensions)

3. L2 Normalization to Obtain Quantum Amplitudes
(Unitary Quantum State in 64-Dimensional Hilbert Space)

Euclidean L2 Normalization

$$c_i = \frac{x_{\text{pad},i}}{\sqrt{\sum_{k=0}^{63} x_{\text{pad},k}^2}}, \quad i = 0, 1, \ldots, 63$$

$$c \in \mathbb{C}^{64} = \begin{bmatrix} c_0 \\ c_1 \\ c_2 \\ c_3 \\ \vdots \\ c_{60} \\ c_{61} \\ c_{62} \\ c_{63} \end{bmatrix}$$

Normalized Quantum Amplitudes
- $c_0, c_1, \ldots, c_{63} \in \mathbb{C}$
- $\sum_{i=0}^{63} |c_i|^2 = 1.0$
- 64-dimensional quantum state
- Encodes 31 features via zero-padding
- Ready for amplitude encoding

$$|\psi\rangle = \sum_{i=0}^{63} c_i |i\rangle, \quad \sum_{i=0}^{63} |c_i|^2 = 1.0$$
Quantum state in $\mathcal{H} = \mathbb{C}^{64}$ (6 qubits)

1. Normalized Feature Amplitudes (31 dimensions)
(Mapping to 6-qubit Computational Basis States)

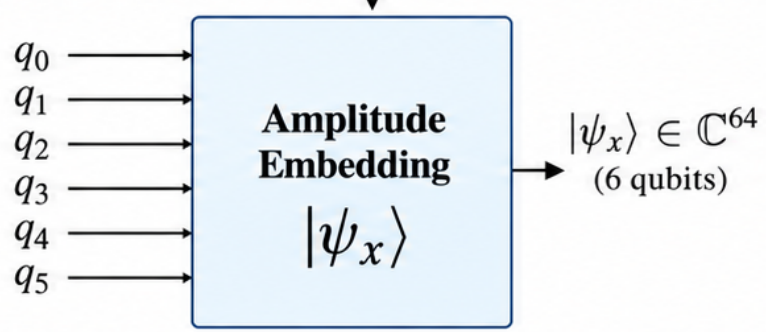
Feature Index (i)	Amplitude c_i (normalized)		6-bit Basis State $\|i\rangle$ (binary)
0	c_0	→	$\|000000\rangle$
1	c_1	→	$\|000001\rangle$
2	c_2	→	$\|000010\rangle$
3	c_3	→	$\|000011\rangle$
4	c_4	→	$\|000100\rangle$
5	c_5	→	$\|000101\rangle$
6	c_6	→	$\|000110\rangle$
7	c_7	→	$\|000111\rangle$
⋮	⋮		⋮
28	c_{28}	→	$\|011100\rangle$
29	c_{29}	→	$\|011101\rangle$
30	c_{30}	→	$\|011110\rangle$

2. Amplitude Embedding into a 6-Qubit Quantum State

Quantum State Preparation

$$|\psi_x\rangle = \sum_{i=0}^{30} c_i\,|i\rangle + \sum_{k=31}^{63} 0\,|k\rangle$$

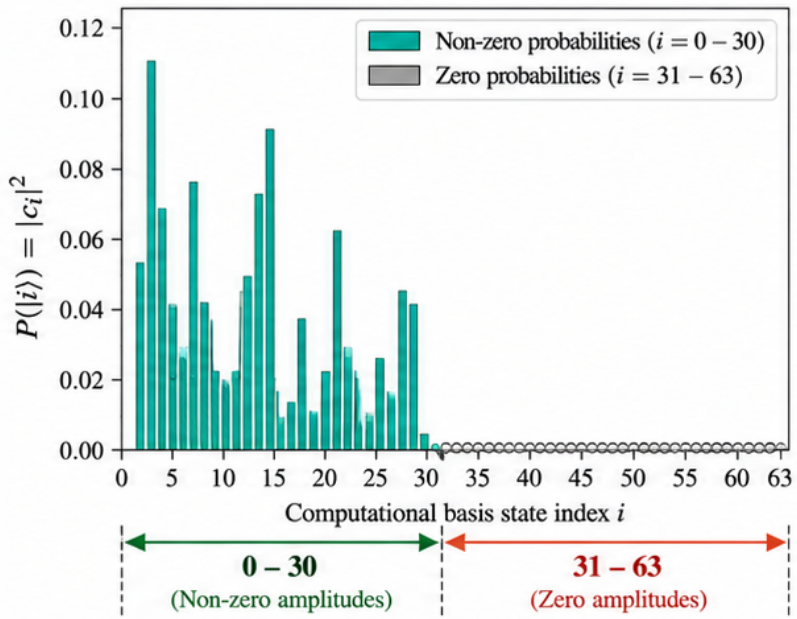
$$c_i \in \mathbb{R}, \quad \sum_{i=0}^{30} |c_i|^2 = 1.0$$



Only the first 31 basis states ($|0\rangle$ to $|30\rangle$) have non-zero amplitudes. States $|31\rangle$ to $|63\rangle$ are zero due to padding.

3. Born Probability Distribution in Computational Basis

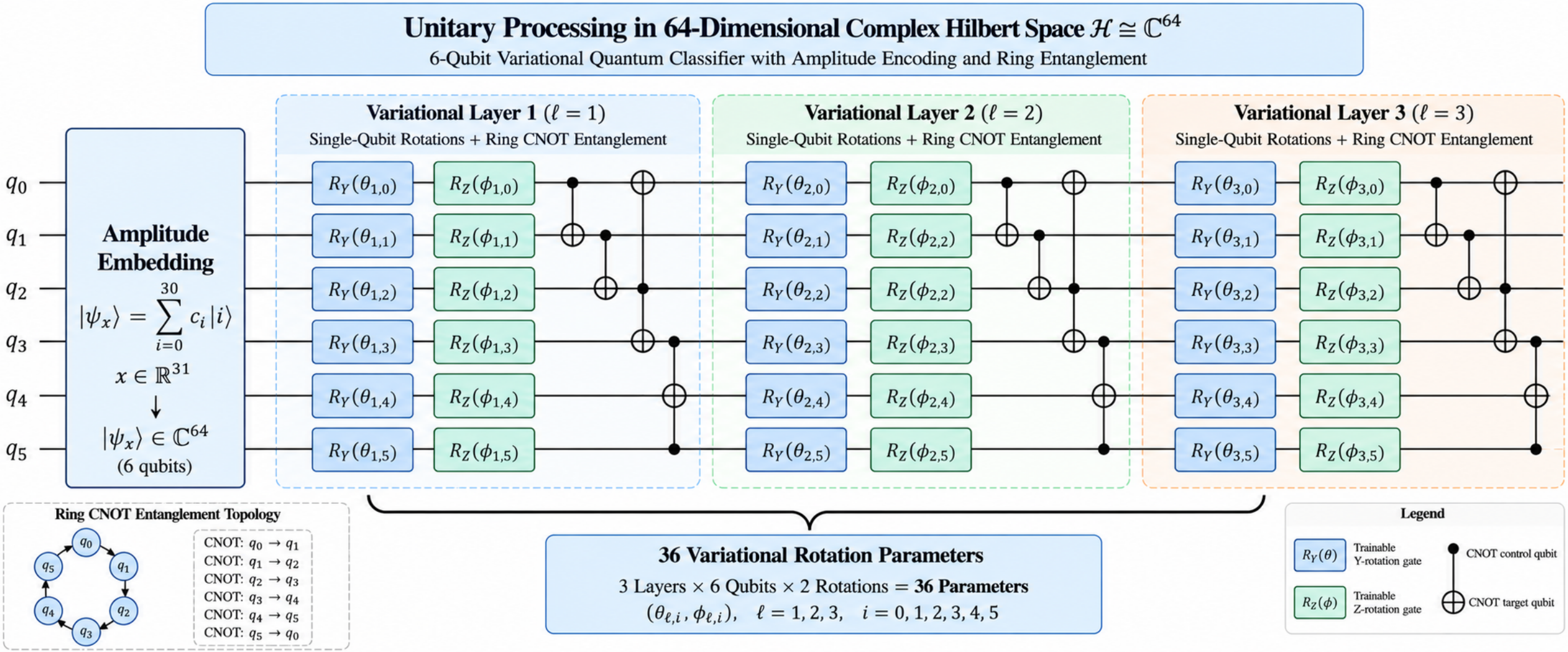
$P(|i\rangle) = |c_i|^2$ for $i = 0, 1, \ldots, 63$

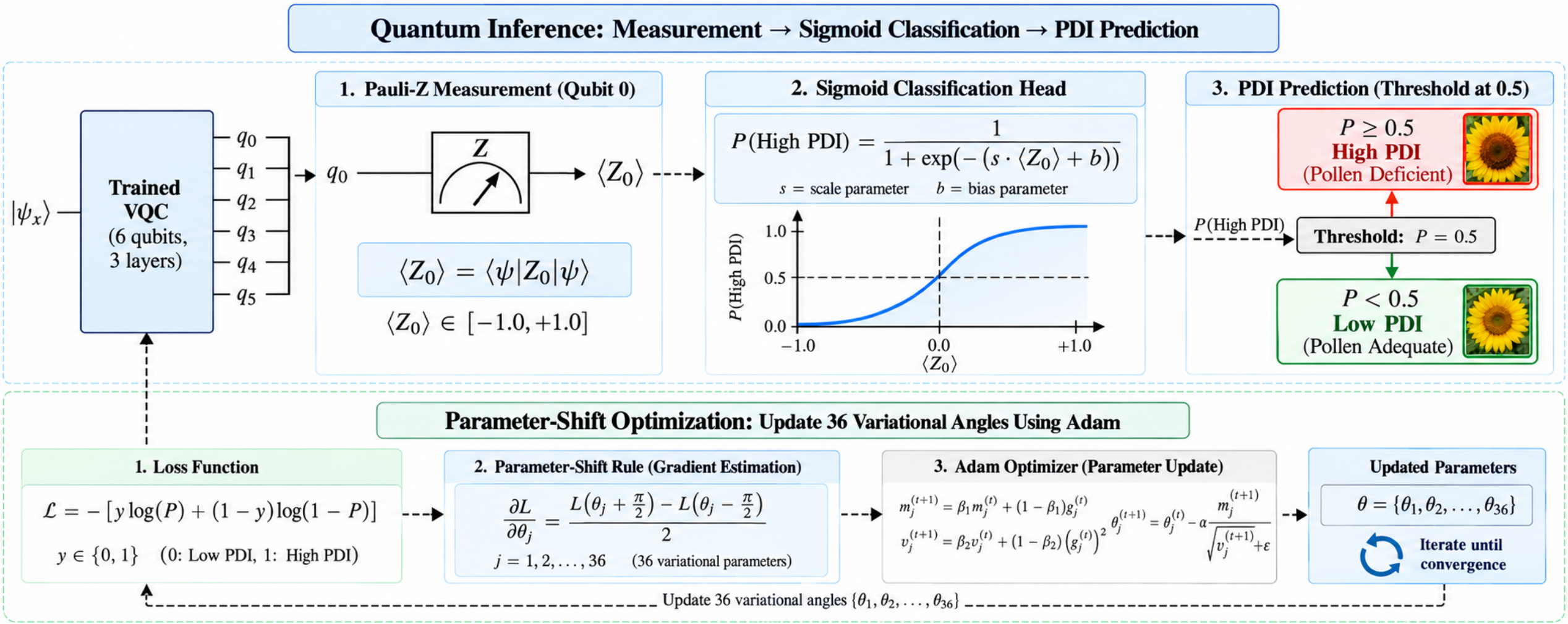


Non-zero states:
$|0\rangle$ to $|30\rangle$
(31 states)

Zero-padded states:
$|31\rangle$ to $|63\rangle$
(33 states)

Zero PCA: 100% of Raw Feature Information Preserved
(All 31 extracted features are directly encoded into the quantum state)


Unitary Processing in 64-Dimensional Complex Hilbert Space $\mathcal{H} \cong \mathbb{C}^{64}$
6-Qubit Variational Quantum Classifier with Amplitude Encoding and Ring Entanglement
Variational Layer 1 ($\ell = 1$)
Single-Qubit Rotations + Ring CNOT Entanglement
Variational Layer 2 ($\ell = 2$)
Single-Qubit Rotations + Ring CNOT Entanglement
Variational Layer 3 ($\ell = 3$)
Single-Qubit Rotations + Ring CNOT Entanglement
q_0
q_1
q_2
q_3
q_4
q_5
Amplitude Embedding
$|\psi_x\rangle = \sum_{i=0}^{30} c_i |i\rangle$
$x \in \mathbb{R}^{31}$
$|\psi_x\rangle \in \mathbb{C}^{64}$
(6 qubits)
$R_Y(\theta_{1,0})$ $R_Z(\phi_{1,0})$
$R_Y(\theta_{1,1})$ $R_Z(\phi_{1,1})$
$R_Y(\theta_{1,2})$ $R_Z(\phi_{1,2})$
$R_Y(\theta_{1,3})$ $R_Z(\phi_{1,3})$
$R_Y(\theta_{1,4})$ $R_Z(\phi_{1,4})$
$R_Y(\theta_{1,5})$ $R_Z(\phi_{1,5})$
$R_Y(\theta_{2,0})$ $R_Z(\phi_{2,0})$
$R_Y(\theta_{2,1})$ $R_Z(\phi_{2,2})$
$R_Y(\theta_{2,2})$ $R_Z(\phi_{2,2})$
$R_Y(\theta_{2,3})$ $R_Z(\phi_{2,3})$
$R_Y(\theta_{2,4})$ $R_Z(\phi_{2,4})$
$R_Y(\theta_{2,5})$ $R_Z(\phi_{2,5})$
$R_Y(\theta_{3,0})$ $R_Z(\phi_{3,0})$
$R_Y(\theta_{3,1})$ $R_Z(\phi_{3,1})$
$R_Y(\theta_{3,2})$ $R_Z(\phi_{3,2})$
$R_Y(\theta_{3,3})$ $R_Z(\phi_{3,3})$
$R_Y(\theta_{3,4})$ $R_Z(\phi_{3,4})$
$R_Y(\theta_{3,5})$ $R_Z(\phi_{3,5})$
Ring CNOT Entanglement Topology
q_0 q_1 q_2 q_3 q_4 q_5
CNOT: $q_0 \to q_1$
CNOT: $q_1 \to q_2$
CNOT: $q_2 \to q_3$
CNOT: $q_3 \to q_4$
CNOT: $q_4 \to q_5$
CNOT: $q_5 \to q_0$
36 Variational Rotation Parameters
3 Layers × 6 Qubits × 2 Rotations = 36 Parameters
$(\theta_{\ell,i}, \phi_{\ell,i})$, $\ell = 1, 2, 3$, $i = 0, 1, 2, 3, 4, 5$
Legend
$R_Y(\theta)$ Trainable Y-rotation gate
$R_Z(\phi)$ Trainable Z-rotation gate
CNOT control qubit
CNOT target qubit


Quantum Inference: Measurement → Sigmoid Classification → PDI Prediction
1. Pauli-Z Measurement (Qubit 0)
2. Sigmoid Classification Head
3. PDI Prediction (Threshold at 0.5)
$|\psi_x\rangle$
Trained VQC (6 qubits, 3 layers)
q_0 q_1 q_2 q_3 q_4 q_5
Z
$\langle Z_0 \rangle = \langle \psi | Z_0 | \psi \rangle$
$\langle Z_0 \rangle \in [-1.0, +1.0]$
$P(\text{High PDI}) = \frac{1}{1 + \exp(-(s \cdot \langle Z_0 \rangle + b))}$
s = scale parameter b = bias parameter
$P \geq 0.5$ High PDI (Pollen Deficient)
Threshold: $P = 0.5$
$P < 0.5$ Low PDI (Pollen Adequate)
Parameter-Shift Optimization: Update 36 Variational Angles Using Adam
1. Loss Function
$\mathcal{L} = -[y \log(P) + (1 - y) \log(1 - P)]$
$y \in \{0, 1\}$ (0: Low PDI, 1: High PDI)
2. Parameter-Shift Rule (Gradient Estimation)
$\frac{\partial L}{\partial \theta_j} = \frac{L(\theta_j + \frac{\pi}{2}) - L(\theta_j - \frac{\pi}{2})}{2}$
$j = 1, 2, \ldots, 36$ (36 variational parameters)
3. Adam Optimizer (Parameter Update)
$m_j^{(t+1)} = \beta_1 m_j^{(t)} + (1 - \beta_1) g_j^{(t)}$
$v_j^{(t+1)} = \beta_2 v_j^{(t)} + (1 - \beta_2) (g_j^{(t)})^2$
$\theta_j^{(t+1)} = \theta_j^{(t)} - \alpha \frac{m_j^{(t+1)}}{\sqrt{v_j^{(t+1)}} + \varepsilon}$
Updated Parameters
$\theta = \{\theta_1, \theta_2, \ldots, \theta_{36}\}$
Iterate until convergence
Update 36 variational angles $\{\theta_1, \theta_2, \ldots, \theta_{36}\}$

Functional Requirements

- **Data ingestion** – load the 5-class sunflower image dataset (5,350 images)
- **Preprocessing** – resize, Lab-space CLAHE contrast enhancement, bilateral denoising
- **Segmentation** – GrabCut + K-means to isolate the flower head and dark disc
- **Feature extraction** – HSV histograms, GLCM texture, shape descriptors, disc ratio
- **PDI proxy labeling** – rule-based Low / Medium / High deficiency
- **Classical modelling** – Random Forest and SVM for growth stage and PDI
- **Dimensionality reduction** – PCA to 6 components for qubit encoding
- **Quantum encoding & VQC** – angle encoding, trainable variational circuit
- **Hybrid head** – sigmoid read-out of ⟨Z⟩ giving P(High deficiency)
- **Evaluation & visualization** – accuracy, F1, confusion matrices, noise sweep, PDI heatmap

Python Libraries and Dependencies Required

1. Quantum Computing
- **PennyLane (PennyLane)**: The core quantum library used to build the 6-qubit Variational Quantum Circuit (VQC), encode the data into angles, and run the noisy density-matrix simulations.

2. Computer Vision & Image Processing
- **OpenCV (opencv-python)**: Used to load the raw images, resize them, convert them to Lab/HSV color spaces, and apply the GrabCut segmentation to isolate the flower.
- **Scikit-Image (scikit-image)**: Used specifically to calculate the GLCM (Gray-Level Co-occurrence Matrix) for the 4 texture features (Contrast, Homogeneity, Energy, Correlation).

3. Classical Machine Learning
- **Scikit-Learn (scikit-learn)**: The powerhouse library used for all classical Machine Learning steps. It handled the Train/Test dataset splitting, K-Means clustering, PCA dimensionality reduction, and the Random Forest & SVM model training.

4. Data Processing
- **Pandas (pandas)**: Used to structure the extracted features into tabular datasets (CSV files), manage the labels, and handle data manipulation.
- **NumPy (numpy<2.0)**: Used for high-speed mathematical array operations behind the scenes (especially for scaling the data and processing matrix equations).

5. Data Visualization
- **Matplotlib (matplotlib)**: Used to generate all the 2D plots (like the Quantum training curves and the PDI heatmaps).
- **Seaborn (seaborn)**: Used on top of Matplotlib to create the beautiful, presentation-ready evaluation charts (the Correlation Heatmap, Pairplot, and the Train/Test bar charts).

6. Configuration
- **PyYAML (PyYAML)**: Used to read the .yaml configuration files, which allowed us to cleanly set parameters (like random seeds, image sizes, and PCA components) without hardcoding them into the python scripts.

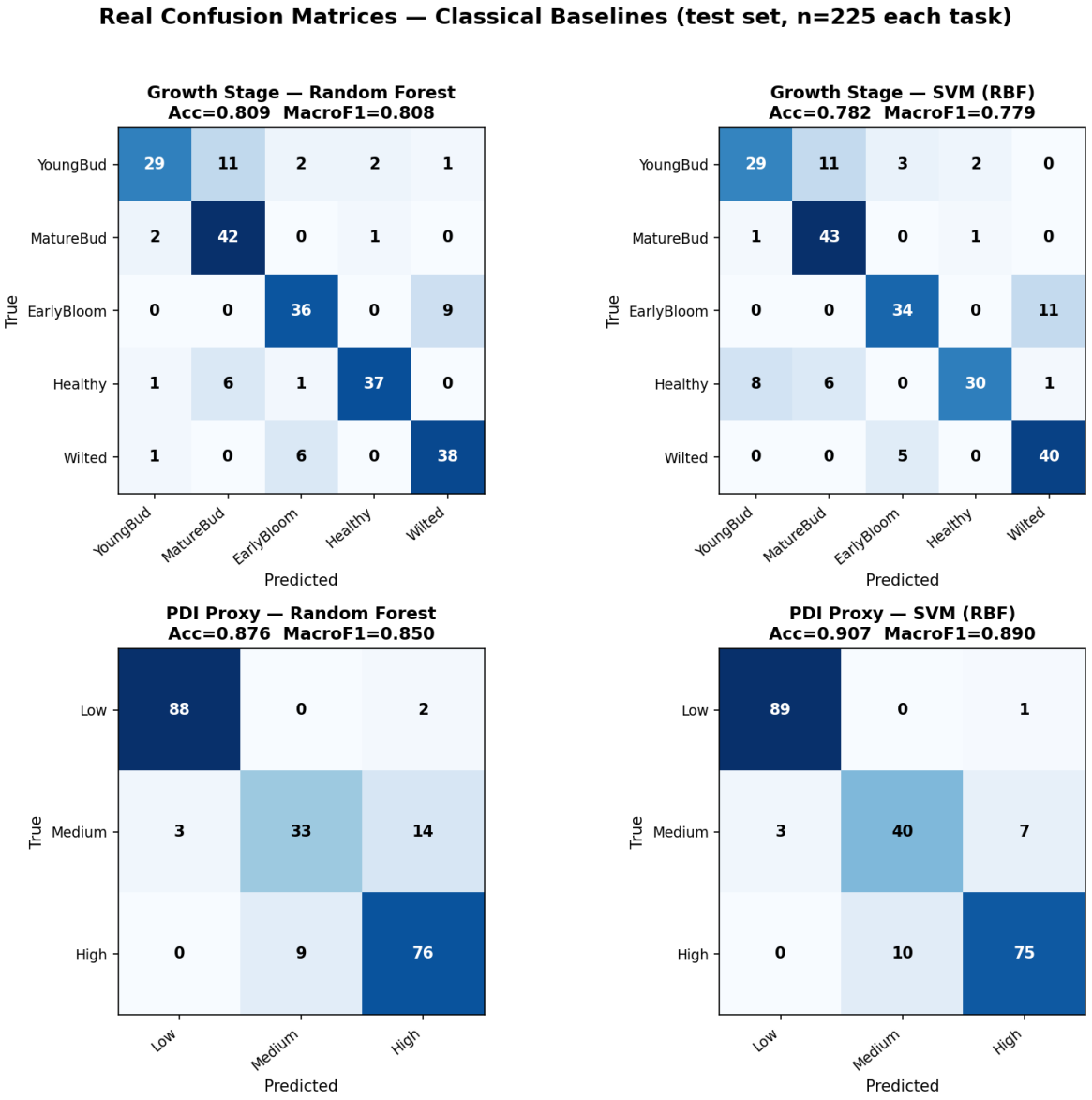
Modules

- **M1: Preprocessing:** resize 256×256, CLAHE, bilateral denoise
- **M2: Segmentation:** GrabCut foreground + K-means disc mask
- **M3: Feature engineering:** 34 handcrafted features per image
- **M4: PDI labeling:** rule-based proxy label from stage, disc and area ratios
- **M5: Classical baseline:** Random Forest / SVM training and evaluation
- **M6: Quantum refinement:** PCA, angle encoding, 6-qubit VQC (PennyLane)
- **M7: Hybrid training:** BCE loss, Adam optimiser, validation-based model selection
- **M8: Evaluation & reporting:** metrics, confusion matrices, noise sweep, PDI heatmap

Experiments and Results – Preprocessing & Segmentation

- **Preprocessing:** resize → Lab CLAHE → bilateral denoise.
- **First attempt (HSV + Otsu):** over-segmented the background (foreground ratio 0.72–0.99).
- **Fix:** GrabCut + K-means gives clean flower / disc masks (foreground ratio 0.08–0.47, disc ratio 0.12–0.21).
- All images shown are real dataset samples (one per class).

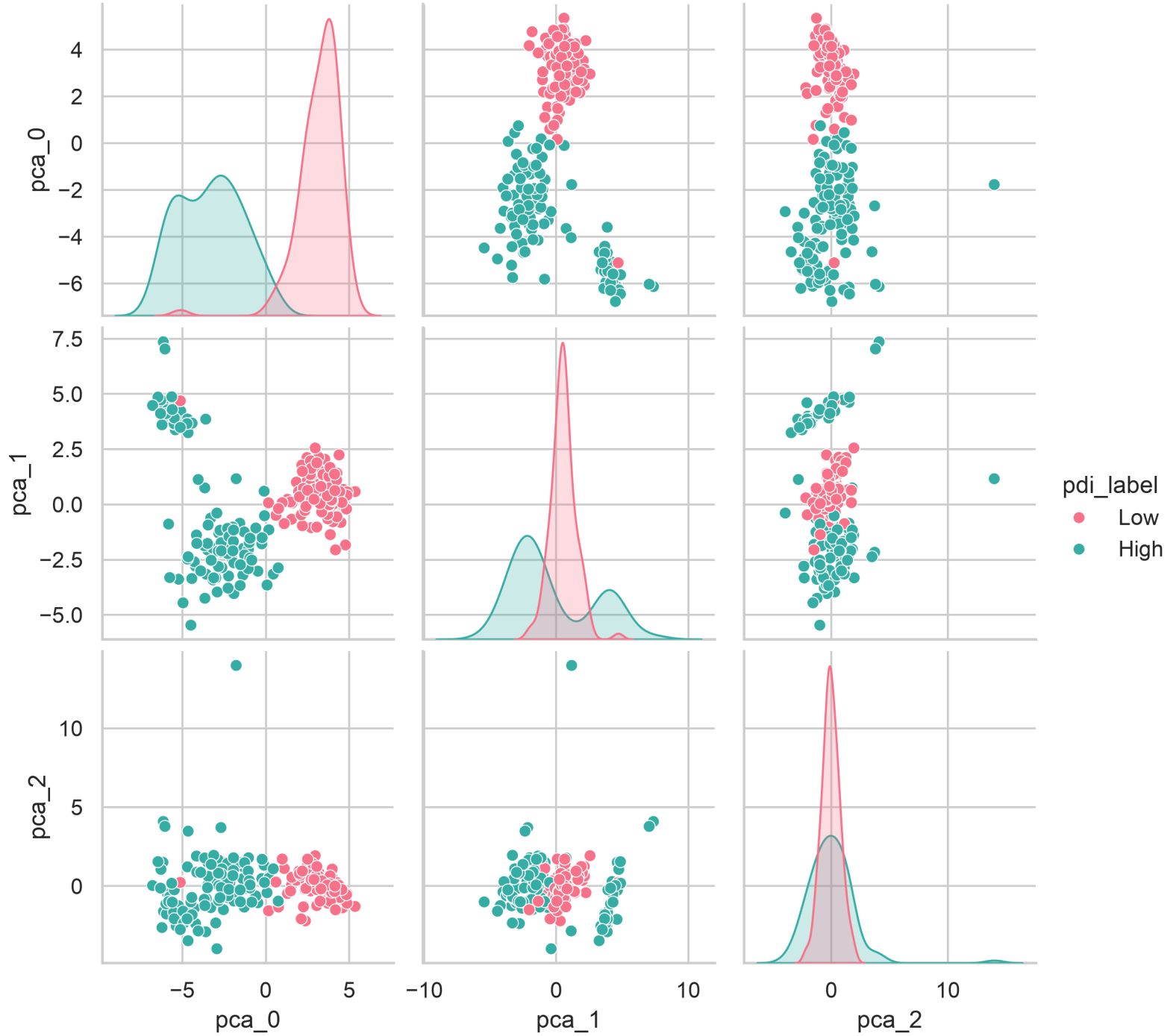
Experiments and Results – Classical Baselines



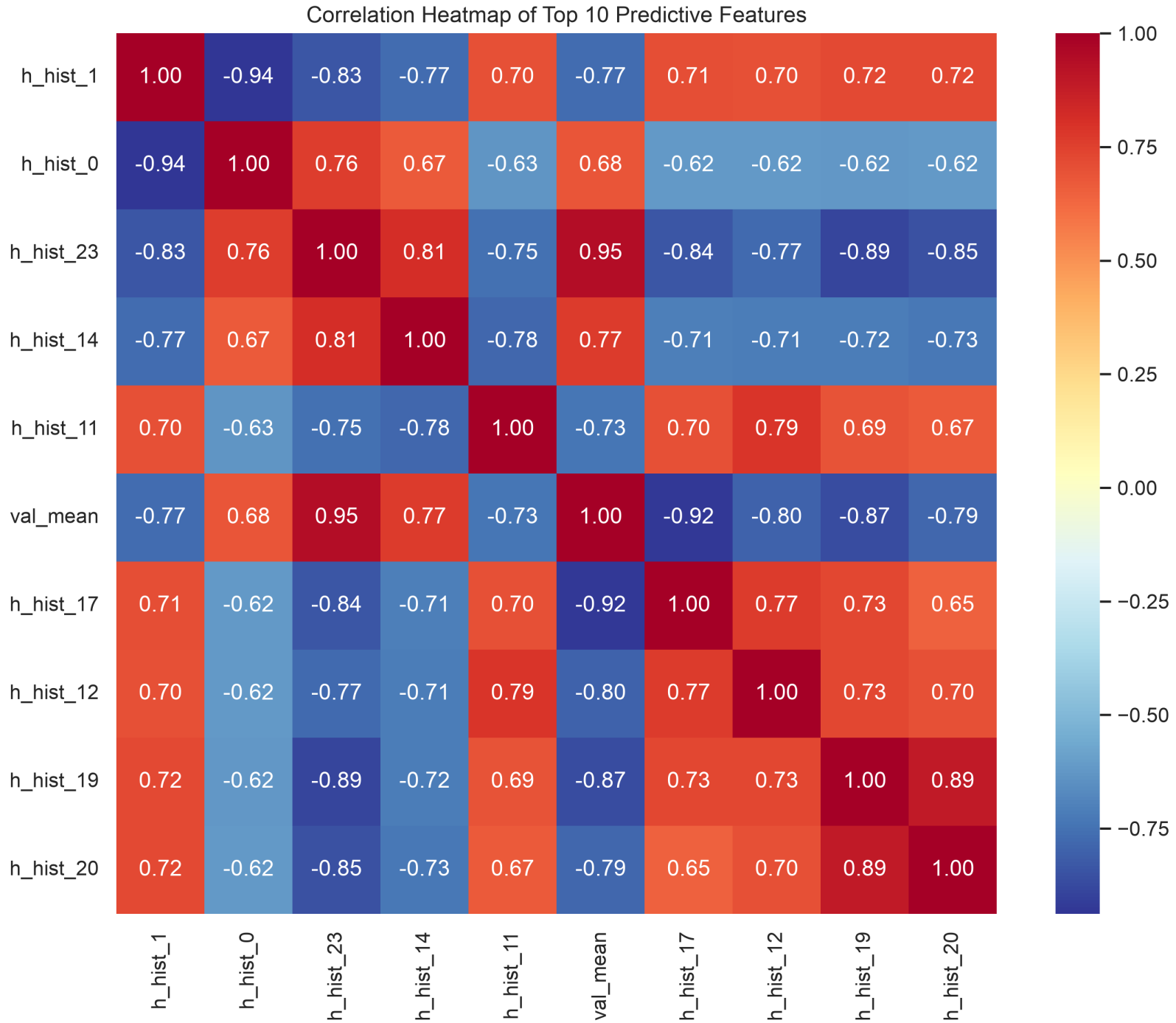
Task	Model	Accuracy	Macro-F1
Growth stage (5-class)	Random Forest	0.809	0.808
Growth stage (5-class)	SVM (RBF)	0.782	0.779
PDI proxy (3-class)	Random Forest	0.876	0.850
PDI proxy (3-class)	SVM (RBF)	0.907	0.890

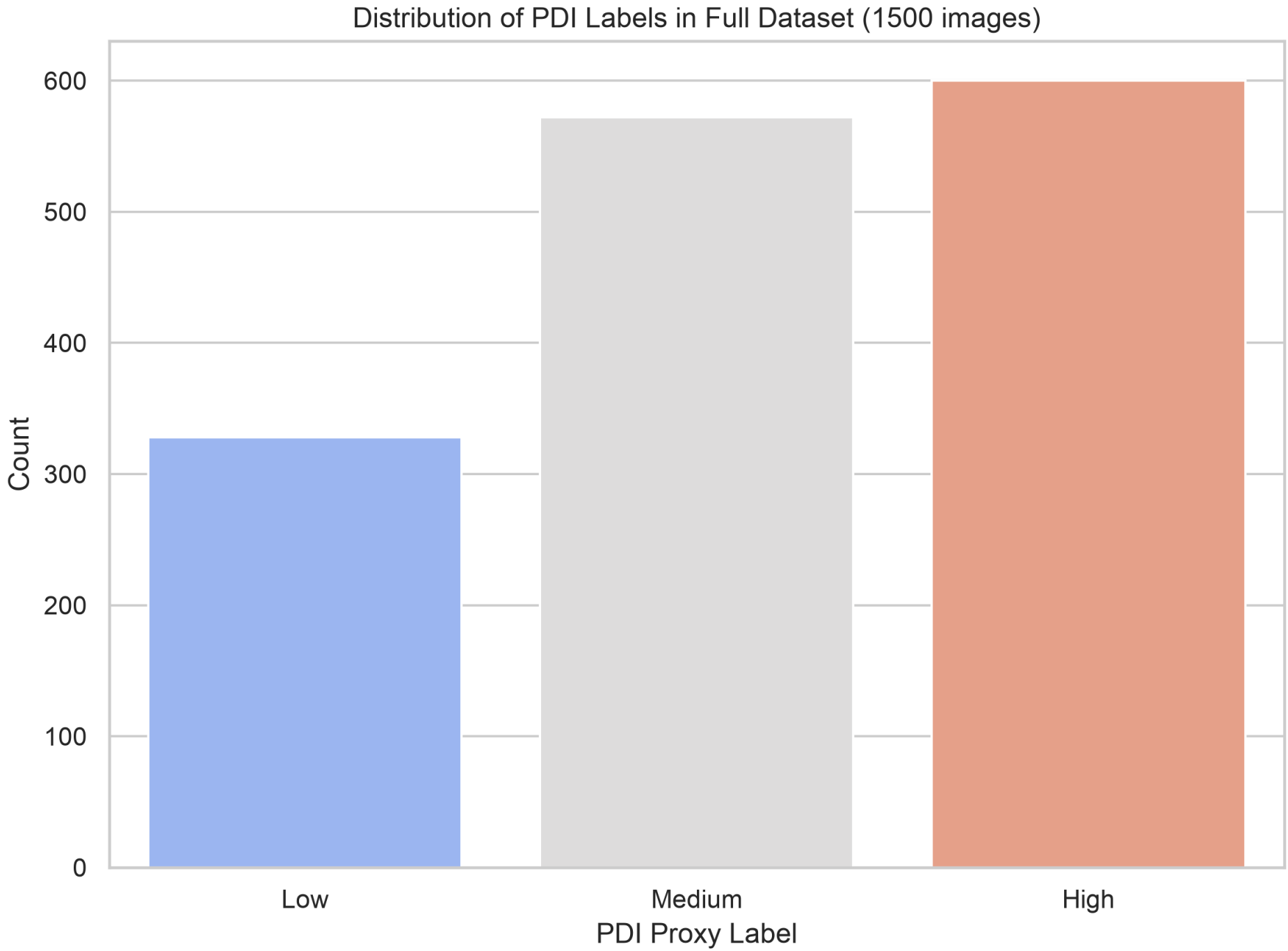
- Held-out test set, n = 225 per task (1,500-image stratified real sample, 70/15/15 split).
- Handcrafted features alone give 78–91% accuracy, far above chance (20% / 33%).
- Main confusions: Healthy ↔ EarlyBloom and Medium ↔ High PDI (visually adjacent stages).

Pair Plot

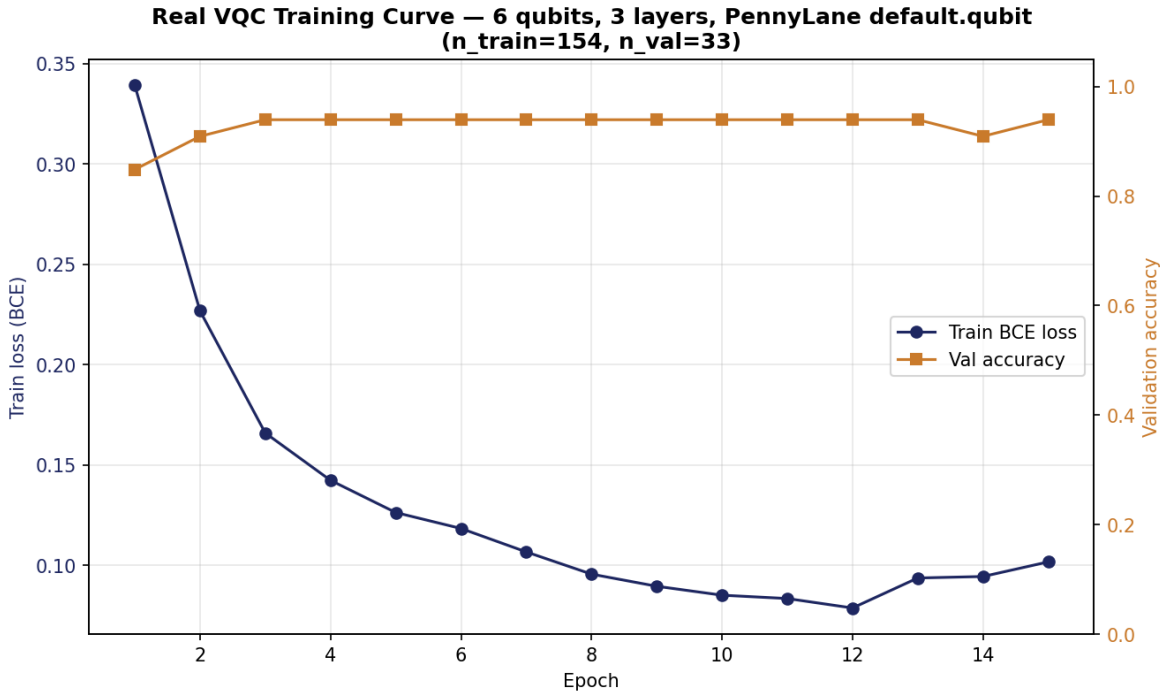


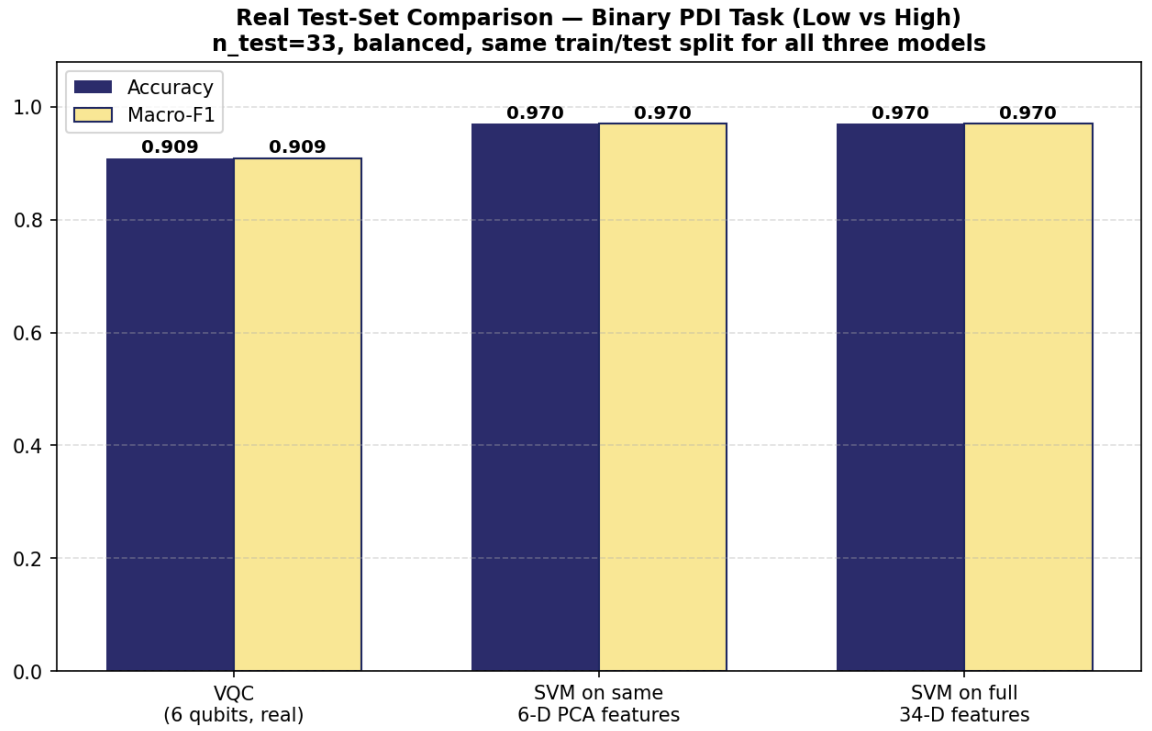
Heatmap


Correlation Heatmap of Top 10 Predictive Features
h_hist_1 1.00 -0.94 -0.83 -0.77 0.70 -0.77 0.71 0.70 0.72 0.72
h_hist_0 -0.94 1.00 0.76 0.67 -0.63 0.68 -0.62 -0.62 -0.62 -0.62
h_hist_23 -0.83 0.76 1.00 0.81 -0.75 0.95 -0.84 -0.77 -0.89 -0.85
h_hist_14 -0.77 0.67 0.81 1.00 -0.78 0.77 -0.71 -0.71 -0.72 -0.73
h_hist_11 0.70 -0.63 -0.75 -0.78 1.00 -0.73 0.70 0.79 0.69 0.67
val_mean -0.77 0.68 0.95 0.77 -0.73 1.00 -0.92 -0.80 -0.87 -0.79
h_hist_17 0.71 -0.62 -0.84 -0.71 0.70 -0.92 1.00 0.77 0.73 0.65
h_hist_12 0.70 -0.62 -0.77 -0.71 0.79 -0.80 0.77 1.00 0.73 0.70
h_hist_19 0.72 -0.62 -0.89 -0.72 0.69 -0.87 0.73 0.73 1.00 0.89
h_hist_20 0.72 -0.62 -0.85 -0.73 0.67 -0.79 0.65 0.70 0.89 1.00
h_hist_1 h_hist_0 h_hist_23 h_hist_14 h_hist_11 val_mean h_hist_17 h_hist_12 h_hist_19 h_hist_20
1.00 0.75 0.50 0.25 0.00 −0.25 −0.50 −0.75


Distribution of PDI Labels in Full Dataset (1500 images)
600
500
400
300
200
100
0
Count
Low
Medium
High
PDI Proxy Label

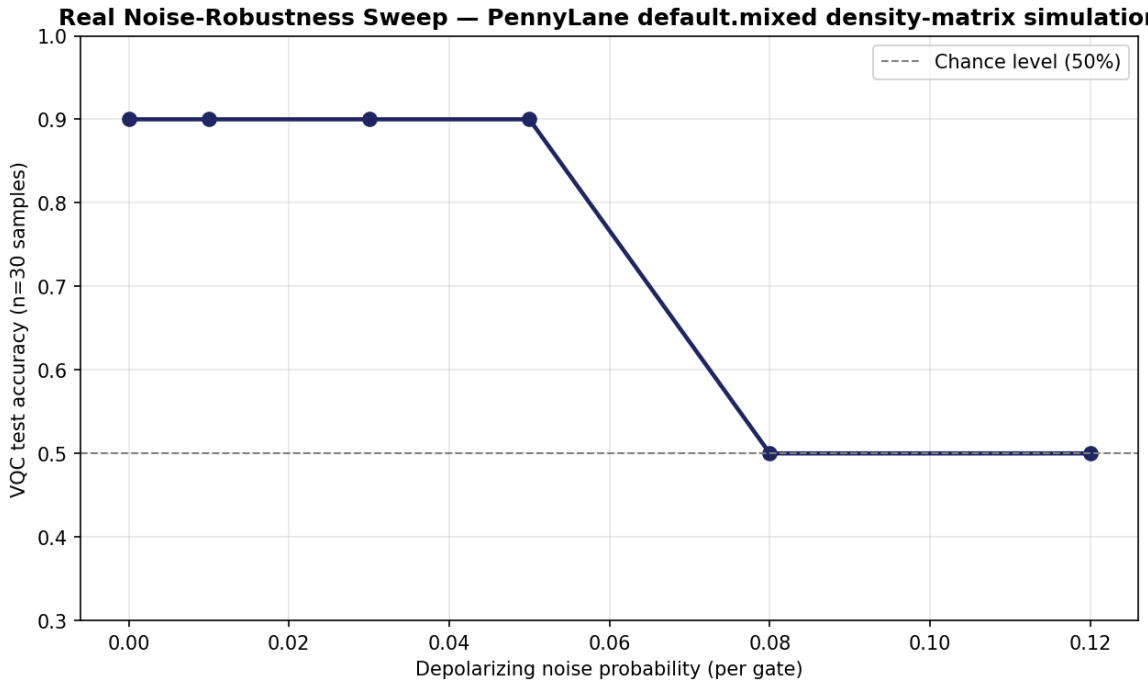
Experiments and Results – Quantum (VQC) vs Classical

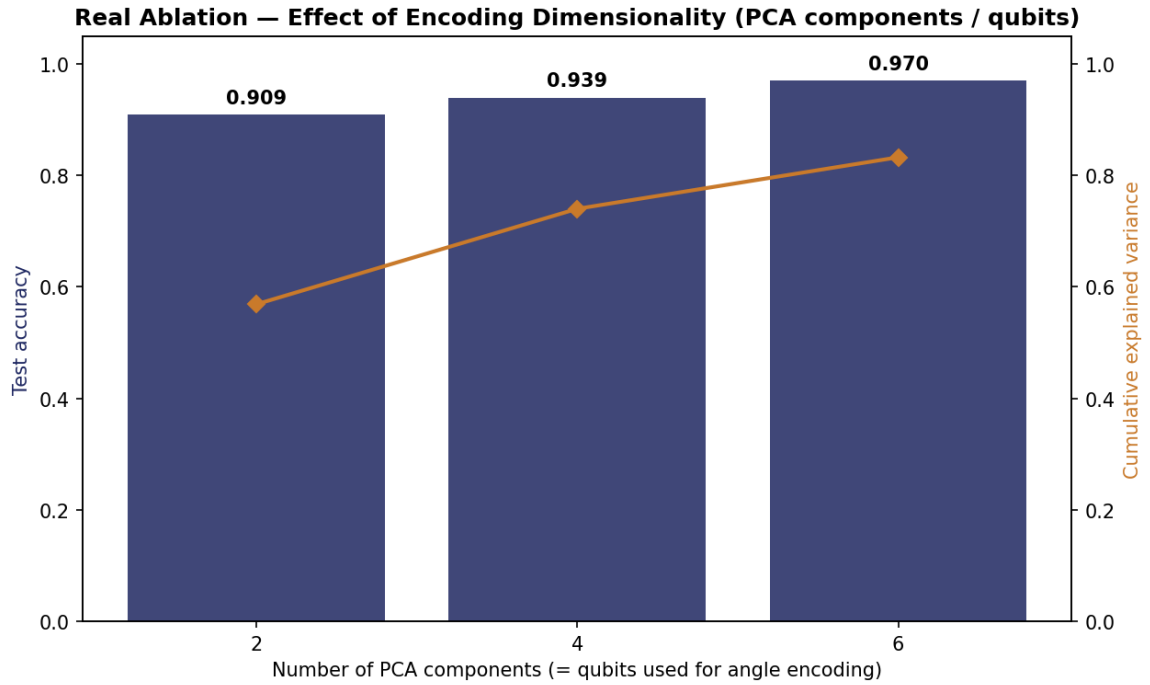




- 6-qubit, 3-layer VQC (PennyLane default.qubit), PCA-6 angle encoding; 220 balanced real images, split 154 / 33 / 33; binary Low vs High PDI.
- **Test accuracy:** VQC 90.9% (F1 0.909) vs SVM on the same 6-D features 97.0% vs SVM on all 34 features 97.0%.
- The classical model is currently ahead at this sample size and qubit budget – reported as measured.

Experiments and Results – Noise Robustness & Ablation





- **Noise:** depolarizing channel after every gate (default.mixed, 30 test samples). Accuracy stays at 90% up to p = 0.05, then falls to chance (50%) at p ≥ 0.08.
- **Ablation:** classical SVM on 2 / 4 / 6 PCA components gives 90.9% / 93.9% / 97.0% – used as a proxy for the qubit-count trade-off; a full per-qubit VQC ablation is future work.

Experiments and Results – PDI Heatmap Output

- **What is shown:** the segmented disc region of real images, coloured by the rule-based PDI proxy label (green = Low, amber = Medium, red = High).
- **Proxy rule:** bud stages → High; EarlyBloom → Medium (Low if disc ratio ≥ 0.30); Healthy → Low (Medium if disc/area too small); Wilted → Medium or High by disc ratio.
- **Note:** the dataset has no manual pollination annotations, so PDI is a documented proxy, not a ground-truth pollination measurement.

Conclusion

- A real 5,350-image sunflower dataset was verified and processed end to end; classical Random Forest / SVM baselines on 34 handcrafted features reach 78–91% accuracy on held-out data.
- A 6-qubit VQC trained on real features reaches 90.9% on the binary PDI task – the hybrid pipeline works, but a classical SVM on the same features is higher (97.0%) at this scale.
- A real noise sweep bounds the NISQ-feasible region at about p = 0.05 depolarizing probability per gate.
- Next: run on the full dataset, add a per-qubit-count VQC ablation and quanvolutional feature maps, and compare statistically for Review 3.

References

[1] R. Jing et al., “Sunflower-YOLO: Detection of sunflower capitula in UAV remote sensing images,” Eur. J. Agronomy, vol. 160, 127332, 2024.

[2] E. Yildirim, I. Colkesen, U. G. Sefercik, “Evaluating transformer- and CNN-based semantic segmentation models for sunflower inflorescence identification,” Precision Agriculture, vol. 27, 56, 2026.

[3] S. B. Luoni et al., “Sunpheno: A deep neural network for phenological classification of sunflower images,” Plants, vol. 13, no. 14, 1998, 2024.

[4] C. Long et al., “Hybrid quantum-classical-quantum convolutional neural networks,” Sci. Rep., vol. 15, 2025.

[5] C. Daka et al., “A novel QCNN framework for quantum-enhanced classification of pixelated colour images,” Sci. Rep., 2026.

[6] C. R. Rice, H. Gan, Z. Wang, “UAV-based approaches for precision-agriculture monitoring,” Drone Syst. Appl., vol. 12, pp. 1–15, 2024.

[7] O. Manasherov, A. Degani, “Multi-agent target allocation and safe trajectory planning for artificial pollination tasks,” Smart Agric. Technol., vol. 8, 100461, 2024.

[8] “Remote phenotyping strategies for sunflower flowering assessments using deep learning approaches,” bioRxiv, 2025, doi:10.1101/2025.04.03.646664.

[9] N. Iamchuen et al., “Automated sunflower head detection and yield estimation from high-resolution UAV imagery using YOLOv11,” Sustainability, vol. 18, no. 2, 1026, 2026.

[10] “Efficient quantum convolutional neural networks for image classification,” arXiv:2505.05957, 2025.

[11] “Hybrid quantum neural network for rice leaf disease classification,” 2024 (venue and authors to be verified).

[12] M. Henderson, S. Shakya, S. Pradhan, T. Cook, “Quanvolutional neural networks: powering image recognition with quantum circuits,” Quantum Mach. Intell., vol. 2, 2, 2020.